Giải Sudoku tổng quát bằng SAT: so sánh ba mã hóa At-Most-One và các đối chứng CP/ILP

Tạ Đình Nguyên[1]

Môn Các vấn đề hiện đại KHMT, INT3301E_4, học kỳ I năm học 2026–2027

ARTICLE INFO

Từ khóa:
Sudoku
SAT
mã hóa At-Most-One
lập trình ràng buộc
lập trình tuyến tính nguyên

TÓM TẮT

Báo cáo đánh giá ba mã hóa At-Most-One trong mô hình Boolean ô–giá trị cho Sudoku tổng quát, cùng các đối chứng CP-SAT, HiGHS và solver thương mại. Ngoài bộ baseline ba lần lặp được báo cáo riêng, phân tích mở rộng bao phủ 16.000 đề và 110.000 lượt chính trong một repetition: 109.664 SAT qua checker và 336 lượt không hoàn tất. Runtime chỉ được tổng hợp trên các giao SAT đã nêu; panel encoding chính dùng 86 đề N25 common-SAT trong một session 32 worker có tranh chấp tài nguyên. Kết quả cho thấy tradeoff runtime theo encoding và cấu trúc profile; báo cáo này không ước lượng repeatability giữa các repetitions, khoảng tin cậy hoặc speedup cô lập. Một witness hợp lệ không xác minh uniqueness; các đề synthetic chia sẻ root và không độc lập.

1. Giới thiệu và nghiên cứu liên quan

Sudoku có luật ngắn gọn nhưng cho phép biểu diễn cùng một bài toán khả thi bằng SAT, CP hoặc ILP. Khi tăng kích thước bảng, số biến, ràng buộc và chi phí dựng mô hình có thể khác nhau giữa các biểu diễn. Vì vậy, đánh giá một cách giải cần phân biệt kích thước mô hình, thời gian dựng/nạp và thời gian giải; số clause nhỏ hơn tự nó chưa chứng minh solver nhanh hơn. Công việc này xét Sudoku tổng quát với $n \geq 2$, $N = n^2$, ưu tiên ảnh hưởng của ba AMO encoding trong cùng mô hình SAT.

1.1. Cơ sở liên quan

Lynce–Ouaknine trình bày biểu diễn Sudoku bằng biến Boolean ô–giá trị và phân biệt mô hình minimal/extended [10]. Bài này là nền tảng mô hình, không cung cấp kết quả cho bộ dữ liệu của dự án. Sinz trình bày sequential unary counter cho ràng buộc cardinality [13]. PySAT cung cấp toolkit và API CardEnc cho các lựa chọn AMO [9, 11]. Dự án giữ cùng ALO/bốn họ EXO, chỉ thay AMO pairwise, seqcounter hoặc bitwise; không đồng nhất bitwise AMO với binary counter trong bài Sinz. Những nguồn này hỗ trợ cách xây dựng, không thay thế kiểm đúng hoặc phép đo của chương trình hiện tại.

1.2. Câu hỏi nghiên cứu

1. Ba AMO encoding ảnh hưởng thế nào đến số biến/phụ, clause và thời gian build/solve/total khi thay kích thước bảng và mật độ clue?
2. Trên đúng tập đề giao nhau, SAT so với CP-SAT và ILP qua HiGHS thế nào về tỷ lệ tìm nghiệm hợp lệ và thời gian trong cùng cutoff?
3. Các đối chứng Gurobi/CPLEX khả dụng trên tập nhỏ cung cấp bằng chứng bổ sung nào, và giới hạn runtime/giấy phép làm giảm độ phủ đến đâu?

S80/B16/C10 chỉ định mẫu so sánh trước phép đo. Mục 12 trả lời các câu hỏi bằng số liệu đã kiểm, luôn lọc đúng tập giao; không ghép thống kê SAT toàn S80 với baseline B16/C10 như cùng một mẫu.

1.3. Phạm vi đóng góp

Đóng góp ở mức bài tập lớn là triển khai mô hình Sudoku tổng quát, tích hợp các encoding đã biết dưới cùng đặc tả, kiểm chứng mô hình/nghiệm và đánh giá trên baseline 906 lượt/ba lần lặp cùng phân tích mở rộng 16.000 đề và 110.000 primary jobs trong một repetition. Kết quả mở rộng giữ riêng giao SAT và giới hạn suy luận theo profile, mẫu số và repetition. Công việc không đề xuất encoding mới, không xem runtime/library sẵn có là thuật toán do tác giả phát minh, và không tuyên bố ưu thế trên mọi n hoặc mọi Sudoku.

Phạm vi kết quả

Báo cáo tách baseline ba lần lặp khỏi phân tích mở rộng repetition 1. Mỗi bảng và hình mở rộng nêu mẫu số, giao dữ liệu và giới hạn diễn giải; thời gian chỉ tính cho kết quả SAT qua checker. Snapshot, receipt, hợp đồng phân tích và bản xuất có hash được lưu cục bộ theo mục Khả dụng mã nguồn và dữ liệu.

[1] Mã sinh viên: 24021587. Báo cáo bài tập lớn số 1. Giảng viên hướng dẫn: TS. Tô Văn Khánh.

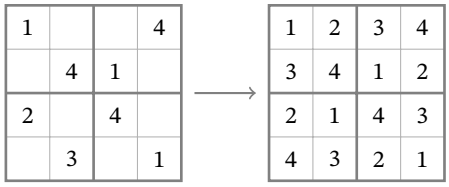
2. Bài toán và ký hiệu

Cho số nguyên $n \geq 2$, đặt $N = n^2$. Sudoku là bảng $N \times N$ chia thành N khối $n \times n$. Hàng và cột được đánh số từ 0; giá trị ô thuộc $D = \{1, \dots, N\}$. Input dùng 0 cho ô trống. Một nghiệm điền đủ bảng, giữ các clues, và chứa mỗi giá trị đúng một lần trong từng hàng, cột và khối. Không yêu cầu nghiệm duy nhất.

Đặt $I = \{0, \dots, N-1\}$, $J = \{0, \dots, n-1\}$ và $G \subseteq I \times I \times D$ là tập các clue khác 0, với $g = |G|$. Biến Boolean $x_{r,c,v}$ đúng khi ô (r, c) mang giá trị v. Có N^3 biến chính. Mapping của chương trình là

$$\mathrm{id}(r, c, v) = (rN + c)N + v \ \in \{1, \dots, N^3\}. \tag{1}$$

Parser từ chối cấu trúc/miền sai bằng INVALID_INPUT. Clues mâu thuẫn nhưng đúng cấu trúc vẫn được mã hóa; tính không khả thi là UNSAT sau khi được chứng minh. Hình 1 minh họa $n = 2$.



Hình 1: Ví dụ 4×4: input (trái) và một cách hoàn thành (phải) dựng để minh họa; không phải output solver hoặc chứng nhận uniqueness.

3. Mô hình SAT và ba AMO encoding

Cách biểu diễn ô–giá trị và mô hình mở rộng của Lynce–Ouaknine là nền tảng tham khảo [10]. Với nhóm $Z = (z_1, \dots, z_m)$, ràng buộc Exactly-One được tách thành ALO và AMO:

$$\mathrm{EXO}(Z) = \mathrm{ALO}(Z) \wedge \mathrm{AMO}(Z), \qquad \mathrm{ALO}(Z) = \bigvee_{i=1}^{m} z_i. \tag{2}$$

Bốn họ sau, mỗi họ có N^2 nhóm dài N, cùng các clue unit clauses tạo công thức Sudoku:

$$\bigwedge_{r,c \in I} \mathrm{EXO}\{x_{r,c,v} : v \in D\}, \qquad \text{ô;} \tag{3}$$

$$\bigwedge_{r \in I, v \in D} \mathrm{EXO}\{x_{r,c,v} : c \in I\}, \qquad \text{hàng–giá trị;} \tag{4}$$

$$\bigwedge_{c \in I, v \in D} \mathrm{EXO}\{x_{r,c,v} : r \in I\}, \qquad \text{cột–giá trị;} \tag{5}$$

$$\bigwedge_{p,q \in J, v \in D} \mathrm{EXO}\{x_{pn+u,qn+w,v} : u, w \in J\}, \qquad \text{khối–giá trị;} \tag{6}$$

$$\bigwedge_{(r,c,v) \in G} x_{r,c,v}, \qquad \text{clues.} \tag{7}$$

Như vậy có $4N^2$ nhóm EXO. Code sinh cùng thứ tự nhóm/literal cho mọi encoder; mỗi nhóm thêm ALO trước AMO, cuối cùng thêm clues theo hàng. Không loại clause trùng, không symmetry breaking hay tiền xử lý riêng theo clue.

3.1. Pairwise

$$E_{\mathrm{pw}}(Z) = \bigwedge_{1 \leq i < j \leq m} (\neg z_i \vee \neg z_j). \tag{8}$$

Mọi cặp input không được cùng đúng. Code tạo trực tiếp các cặp, không dùng biến phụ.

3.2. Sequential counter

Theo nguyên lý sequential unary counter [13], có thể đặc tả AMO bằng chuỗi $s_1, \dots, s_{m-1}$:

$$\begin{aligned} E_{\mathrm{sc}}(Z, S) = {} & (\neg z_1 \vee s_1) \wedge (\neg z_m \vee \neg s_{m-1}) \\ & \wedge \bigwedge_{i=2}^{m-1} \big[(\neg z_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg z_i \vee \neg s_{i-1})\big]. \end{aligned} \tag{9}$$

Input đúng làm chuỗi truyền về sau, ngăn một input sau đó cũng đúng. Chiều mở rộng chọn $s_i = \bigvee_{j \leq i} z_j$. Chương trình gọi `CardEnc.atmost` với `bound=1`, `EncType.seqcounter` và `top_id` hiện hành [11, 9]. Biểu thức trên mô tả nguyên lý/semantics, không khẳng định thứ tự literal hay biến phụ trùng nguyên byte với binary PySAT.

3.3. Bitwise AMO

Đặt $k = \lceil \log_2 m \rceil$ và gán mỗi input một mã nhị phân khác nhau $b(i) \in \{0,1\}^k$. Với các selector $a_1, \dots, a_k$, đặt $\ell_{ij} = a_j$ nếu $b(i)_j = 1$, và $\ell_{ij} = \neg a_j$ nếu $b(i)_j = 0$. Khi đó

$$E_{\text{bw}}(Z, A) = \bigwedge_{i=1}^{m} \bigwedge_{j=1}^{k} (\neg z_i \vee \ell_{ij}). \tag{10}$$

Hai input đúng đòi hai mã khác nhau nên xung đột ở ít nhất một bit. Khi có một input đúng, chọn mã của nó; khi không có input đúng, chọn selector tùy ý. Chương trình dùng `EncType.bitwise` qua cùng API AMO. Đây là mã hóa AMO cho các literal one-hot, không phải thay biến ô bằng số nhị phân. Nguồn API/source bitwise được ghi riêng [11, 12]; không gán encoding này cho Sinz.

4. Tính đúng đắn

Gọi $F_e(X, A)$ là CNF cho encoding e, gồm cả clues. Mỗi AMO phải thỏa đặc tả $\exists A_Z\, E_e(Z, A_Z) \iff \sum_i z_i \le 1$. Các cấu trúc trên đáp ứng đặc tả này. Trong code, toàn bộ N^3 ID chính được dự trữ trước; mỗi nhóm cấp biến phụ phía trên `top_id` và cập nhật nó, nên các nhóm không dùng chung biến phụ. Khi đó

$$\exists A\, F_e(X, A) \quad \iff \quad \text{Sudoku}_G(X). \tag{11}$$

Chiều thuận. ALO và AMO của ô cho đúng một giá trị trong mỗi ô. Các nhóm hàng–giá trị, cột–giá trị và khối–giá trị cho mỗi giá trị xuất hiện đúng một lần trong các đơn vị tương ứng. Unit clauses giữ mọi clue. Vì vậy giải mã các biến chính của một model CNF tạo một nghiệm hợp lệ.

Chiều ngược. Từ một nghiệm Sudoku giữ clues, đặt đúng các biến ô–giá trị được chọn; mỗi nhóm có đúng một input đúng. Pairwise đã thỏa; sequential mở rộng bằng các OR tiền tố; bitwise chọn mã input đúng. Do các nhóm có tập phụ rời nhau, ghép các mở rộng cục bộ thành một phép gán A thỏa toàn CNF. Không đòi mọi phép gán phụ đều thỏa, cũng không suy nghiệm là duy nhất.

5. Kích thước lý thuyết và số đếm đã kiểm

Với $m \ge 4$, các cấu trúc AMO được trình bày có số phụ $a_e(m)$ và clause $c_e(m)$ như sau:

e	$a_e(m)$	$c_e(m)$
pairwise	0	$m(m-1)/2$
seqcounter	$m-1$	$3m-4$
bitwise	$\lceil \log_2 m \rceil$	$m\lceil \log_2 m \rceil$

(12)

Do mỗi nhóm còn một ALO, tổng lý thuyết là

$$V_e = N^3 + 4N^2 a_e(N), \qquad C_e = 4N^2[1 + c_e(N)] + g. \tag{13}$$

Pairwise có chính xác N^3 biến và $4N^2[1 + N(N-1)/2] + g$ clause khi giữ trùng. Các cấu trúc cho độ lớn clause lần lượt $\Theta(N^4)$, $\Theta(N^3)$, $\Theta(N^3 \log N)$; cả ba có tổng biến $\Theta(N^3)$, vì $g \le N^2$. Đây là kích thước biểu diễn, không phải giới hạn thời gian solve hoặc bằng chứng encoder nào nhanh hơn.

Bảng 1 được sinh từ `diagnostics/model/results.json`: PySAT 1.9.dev16, $N \in \{4, 9, 16, 25\}$. Đây là số đếm CNF trước preprocessing, không phải benchmark; các ca chẩn đoán có $g = 2$ và bảng trừ hai clue unit để hiển thị $C - g$. Seqcounter/bitwise quan sát được khớp các biểu thức trên tại bốn kích thước này. Không suy mọi biến thể/thư viện hay mọi độ dài input có cùng count; API có thể có shortcut ở nhóm nhỏ. Source upstream đã đọc chưa được coi là bằng chứng binary đang chạy.

Bảng 1
Số đếm cấu trúc từ diagnostics; $C - g$ không gồm clue units.

N	AMO	N^3	Phụ	V_e	$C_e - g$
4	pairwise	64	0	64	448
4	seqcounter	64	192	256	576
4	bitwise	64	128	192	576
9	pairwise	729	0	729	11988
9	seqcounter	729	2592	3321	7776
9	bitwise	729	1296	2025	11988
16	pairwise	4096	0	4096	123904
16	seqcounter	4096	15360	19456	46080
16	bitwise	4096	4096	8192	66560
25	pairwise	15625	0	15625	752500
25	seqcounter	15625	60000	75625	180000
25	bitwise	15625	12500	28125	315000

6. Mô hình CP và ILP đối chứng

Các đối chứng mô tả cùng bài toán: tìm một nghiệm giữ clues, không tối ưu độ khó hoặc chất lượng một bảng Sudoku. Phần này đối chiếu các builder đã nghiệm thu; dựng được mô hình không có nghĩa đã giải thành công hay được giấy phép cho phép ở mọi kích thước.

6.1. CP với AllDifferent

Với ký hiệu I, J, D, G đã định nghĩa, đặt một biến nguyên $z_{r,c} \in D$ cho mỗi ô. Ràng buộc là

$$\text{AllDifferent}(z_{r,c} : c \in I), \qquad r \in I, \tag{14}$$

$$\text{AllDifferent}(z_{r,c} : r \in I), \qquad c \in I, \tag{15}$$

$$\text{AllDifferent}(z_{pn+u,qn+w} : u, w \in J), \qquad p, q \in J, \tag{16}$$

$$z_{r,c} = v, \qquad (r, c, v) \in G. \tag{17}$$

Vì mỗi đơn vị có N ô nhận giá trị trong miền N phần tử, AllDifferent buộc mỗi giá trị xuất hiện đúng một lần. Có N^2 biến nguyên và $3N + g$ ràng buộc cấp mô hình; không coi số này là số clause/biến sau biến đổi nội bộ của solver.

Builder CP-SAT tạo `CpModel`, một `CpSolver` mới và đặt một worker [3]. Builder CP Optimizer dùng cùng miền, ràng buộc AllDifferent và clues, với `Workers=1` [8]; executable được kiểm riêng. Các builder chưa thực hiện solve, decode hoặc xác nhận nghiệm trong gói đã bàn giao.

6.2. ILP nhị phân dùng chung

Đặt $y_{r,c,v} \in \{0, 1\}$ cho $r, c \in I$, $v \in D$. Bốn họ đẳng thức và clues là

$$\sum_{v \in D} y_{r,c,v} = 1, \qquad r, c \in I, \tag{18}$$

$$\sum_{c \in I} y_{r,c,v} = 1, \qquad r \in I,\ v \in D, \tag{19}$$

$$\sum_{r \in I} y_{r,c,v} = 1, \qquad c \in I,\ v \in D, \tag{20}$$

$$\sum_{u,w \in J} y_{pn+u,qn+w,v} = 1, \qquad p, q \in J,\ v \in D, \tag{21}$$

$$y_{r,c,v} = 1, \qquad (r, c, v) \in G. \tag{22}$$

Builder xuất ma trận CSR M, vector vế phải toàn 1 và objective toàn 0:

$$\min\ \mathbf{0}^{\mathrm{T}} Y = 0 \quad \text{subject to} \quad MY = \mathbf{1}, \quad 0 \le Y \le 1, \quad Y \in \mathbb{Z}^{N^3}. \tag{23}$$

Do mọi nghiệm hợp lệ có cùng objective 0, đây là bài toán khả thi. Không tối thiểu hóa thời gian chạy bằng objective này. Cột zero-based là $\mathrm{id}(r, c, v) - 1$. Mô hình có N^3 biến nhị phân, $4N^2 + g$ hàng đẳng thức và $4N^3 + g$ phần tử khác 0, trước mọi preprocessing. Mỗi nhóm EXO/clue tạo một hàng; giữ cả các hàng có thể dư thừa, không giảm miền hoặc pruning riêng theo backend.

Cùng cấu trúc ILP được dùng cho các adapter Gurobi, CPLEX MIP và HiGHS. Luồng thực thi, thiết lập luồng/ngân sách, trạng thái và checker được mô tả riêng ở Mục 7; bằng chứng builder không tự chứng minh độ phủ runtime. Về mô hình, liên hệ $y_{r,c,v} = 1 \iff z_{r,c} = v$ cho cùng tập nghiệm Sudoku như biểu diễn SAT, dù cách solver biểu diễn nội bộ khác nhau.

7. Thuật toán và triển khai adapter

Giả mã dưới đây tóm tắt luồng đã triển khai bằng parser, model builder, solver adapter, decoder và checker riêng. Các công thức ở phần trước là đặc tả mô hình; source code thực nằm trong repository. Giả mã không thay cho runner có hard process guard và không chứng minh một đợt benchmark đã chạy.

7.1. Cách gọi thư viện và kiểm nghiệm

Backend SAT là `pysat.solvers.Glucose3` trong python-sat 1.9.dev16 [9, 1, 2]. Năm đối chứng dùng OR-Tools CP-SAT [3], gurobipy [4], CPLEX Python API trực tiếp (`cplex.Cplex`, không dùng DOcplex.MP) [7], DOcplex.CP cho CP Optimizer [8], và highspy cho HiGHS [5, 6]. SciPy chỉ cung cấp ma trận sparse cho ILP trong các adapter này, không phải backend solve HiGHS qua `scipy.optimize.milp`. Phía SAT, API công khai là `prepare_backend` và `solve_sat`; mỗi lượt dùng Glucose3 mới. Adapter gọi CNF builder với encoding đã chọn, dùng native solve có hỗ trợ interrupt, rồi chuyển model qua decoder và `validation.check_solution`. Decoder không suy giá trị ô từ biến phụ.

Phía đối chứng, `solve_baseline` nhận một trong các method `cpsat`, `gurobi`, `cplex_mip`, `cpoptimizer`, `highs`. CP-SAT/CP Optimizer dùng builder CP; Gurobi/CPLEX MIP/HiGHS nhận cùng ma trận ILP, bounds, integrality và objective hằng. Mỗi lượt có đối tượng mới và một luồng/worker. Các adapter giữ raw status và dùng cùng checker production.

Thuật toán 1 Giải một Sudoku và kiểm nghiệm kết quả

1. Đọc/parse input thành Puzzle; lỗi cấu trúc, JSON hoặc file trả INVALID_INPUT. Kiểm method/encoding/budget theo API trước khi dựng mô hình.
2. Chuẩn bị thư viện backend ngoài phần đo; runtime thiếu được báo UNAVAILABLE.
3. Bắt đầu đồng hồ đơn điệu t_0; lấy deadline sớm hơn giữa t_0+cutoff và deadline caller (nếu có). Nếu hết ngân sách thì TIMEOUT.
4. Dựng CNF hoặc model CP/ILP và tạo/nạp solver mới. Tính phần build/setup; nếu đã hết ngân sách thì bỏ qua solve và trả TIMEOUT.
5. Gọi solver với ngân sách còn lại, dùng giới hạn native và cơ chế ngắt hợp tác được API hỗ trợ. Giữ raw status.
6. Chốt thời điểm kết thúc native solve và các thời gian đã đo. Kết quả đến muộn thành TIMEOUT; không nhận incumbent đến muộn.
7. Nếu chứng minh không khả thi, trả UNSAT. Nếu không có candidate hợp lệ, phân loại TIMEOUT, ERROR hoặc LICENSE_LIMIT theo bằng chứng; không biến unknown thành UNSAT.
8. Giải mã candidate thành bảng đầy đủ; lỗi decode là ERROR. SAT chỉ giải mã biến chính; MIP phải qua kiểm tính hữu hạn/nguyên trước khi chọn giá trị.
9. Chạy checker độc lập kiểm miền, hàng, cột, khối và clues. Chỉ trả SAT kèm nghiệm nếu checker chấp nhận; từ chối hoặc lỗi checker là ERROR.
10. Trên mọi đường thoát, dừng/join timer trước khi hủy native solver và dọn tài nguyên. Lỗi cleanup được báo rõ; không công bố nghiệm thành công khi cleanup lỗi.

MIP chỉ chấp nhận giá trị hữu hạn cách 0 hoặc 1 không quá 10^{-6}, với đúng một giá trị được chọn mỗi ô; không làm tròn tùy ý một nghiệm phân số. HiGHS còn phải báo primal feasibility/integrality phù hợp trước decode. Một incumbent trong lượt time-limit chỉ có thể thành SAT nếu đến đúng hạn và checker chấp nhận. UNSAT cần bằng chứng infeasibility; thiếu runtime, vượt license và unknown được phân biệt.

7.2. Ranh giới thời gian và bảo vệ tiến trình

Imports/đọc input nằm ngoài phần đo. Build gồm dựng mô hình/CNF, tạo và nạp solver; solve dùng phần ngân sách còn lại. Decode, checker và cleanup nằm ngoài build+solve. Việc nhận nghiệm theo deadline dựa vào thời điểm native solve kết thúc, không dùng thời điểm checker hoàn tất để tính thời gian solve. Các thành phần thời gian không biết sau khi bị dừng cưỡng bức phải để thiếu, không bịa số.

Ngắt native của SAT và timer/giới hạn native của baseline là hợp tác. Riêng DOcplex 2.32.264, TimeLimit nằm trong CPO đã gửi, nên không hứa chỉnh chính xác native TimeLimit sau load: adapter dùng preload để tính build, timer gọi abort_search theo deadline thực và từ chối kết quả muộn. Preload hook riêng cần kiểm lại khi nâng DOcplex. Nếu timer join thất bại, adapter báo ERROR và có thể hoãn hủy native để tránh callback vào bộ nhớ đã giải phóng.

Measurement-end hook và hard worker/process-tree guard đã được nghiệm thu. Runner dùng START/END để áp cutoff đúng phần đo, rồi chuyển sang watchdog hậu xử lý và dọn process tree khi cần. Pilot đã kiểm đường tích hợp này; native timeout riêng vẫn là hợp tác. Mục 9 mô tả cấu hình đợt chính đã khóa; Mục 12 báo kết quả thực chạy, tách khỏi pilot.

8. Protocol của thí nghiệm mở rộng

Bảng 2
Mẫu số mục tiêu theo cấu hình; chưa phải coverage quan sát.

N	Đề mục tiêu	Cấu hình	Lượt chính
9	10,000	8	80,000
16	5,000	5	25,000
25	1,000	5	5,000
Tổng	16,000		110,000

Bảng nhân quota đề với cấu hình áp dụng theo N cho một repetition. Nó không thay observed count. ERROR/TIMEOUT vẫn là primary rows trong coverage; cấu hình không áp dụng không sinh job. Kết quả cuối cần tách planned, selected, observed, solved denominators.

Ma trận mở rộng cố định ba kích thước: N=9 public, N=16 synthetic và N=25 synthetic. Mục tiêu là 16.000 đề và 110.000 lượt chính trong repetition 1; repetitions 2/3 của lịch sử cũ nằm ngoài mẫu số này. Đây là mục tiêu, không phải khẳng định mọi job thành công hoặc mọi đề được giải. Cutoff chính 10 giây tính từ START bao gồm dựng/nạp mô hình và solve. Tập công cụ theo N được khóa trước đo. Ba AMO encoding dùng cùng Glucose3 trên tập giao hợp lệ; CP-SAT/HiGHS là baseline mở rộng, còn solver thương mại chỉ ở cấu hình kích thước nhỏ được phép. Không áp dụng ở N lớn không đồng nghĩa thất bại hoặc timeout. License/unavailable giữ trạng thái riêng. Đề public giữ nguồn, nhãn gốc và hash; synthetic giữ root, family, mask clue và liên hệ witness. Actual clue count có thể khác tỷ lệ mục tiêu do làm tròn. Dedup nội dung không chứng minh uniformity, uniqueness, độ khó hoặc khác lớp đẳng cấu. Năm mask chung root là cluster, không phải năm quan sát độc lập. Session receipt khóa profile, worker count và timing policy. Không gán worker mới ngược về session cũ. Session tuần tự và song song trình bày riêng; nếu dùng ID khác nhau, không suy speedup. Mọi paired timing đòi cùng ID và profile/session phù hợp, đồng thời nêu matched count và trạng thái cả hai phía. Chọn primary theo manifest, không chọn batch hoặc repetition

nhanh nhất. Snapshot SQLite đóng và nhất quán là nguồn phân tích. RUNNING/INTERRUPTED, logs/ETA, lịch tương lai và diagnostic retry không thành primary. Status primary không bị thay bằng lần chạy chẩn đoán hoặc thay mẫu số. Coverage đếm mọi trạng thái primary; runtime chỉ tóm tắt SAT qua checker. TIMEOUT là censoring tại cutoff, không phải UNSAT hay runtime chính xác. Thành phần thời gian chưa biết giữ null. Nếu có PAR-2 thì công thức/denominator riêng, không trộn vào runtime. Một repetition không có median ba lượt hoặc repeatability. Phân vị giữa puzzle trong stratum là biến thiên giữa instance, không giữa repetitions. Public N9 và synthetic N16/N25 khác cơ chế lấy mẫu; size không được cô lập nhân quả. Rating public chưa hiệu chuẩn. Generator synthetic có bias và cấu trúc root; clue fraction không đơn điệu với độ khó. Một witness hợp lệ không chứng minh uniqueness. Kết luận chỉ áp cho tập giao, profile, phiên bản và máy được receipt ghi.

9. Thiết lập thực nghiệm

Đợt chính `main-20261016-01` đã thực hiện đủ 906 lượt mới với cutoff 10 giây. Dataset, source code, cấu hình và dependency đã khóa; `run.json` và `execution-freeze.json` lưu metadata/hash, được đối chiếu với audit analysis final. Các kết quả ở Mục 12 chỉ thuộc đợt chính này.

9.1. Máy và runtime

Các lượt được cấu hình chạy tuần tự trên cùng máy Windows x64, một job tại một thời điểm và một luồng/worker mỗi engine. Bảng 3 tách binding khỏi native runtime khi có bằng chứng. Không suy phiên bản native Glucose3 từ PySAT, hoặc HiGHS nhúng SciPy từ highspy.

Bảng 3
Máy và phiên bản trong metadata đợt chính đã khóa.

Thành phần	Giá trị đã ghi
OS	Windows 11, 10.0.26300, x64
CPU	AMD Ryzen 7 H 260 w/ Radeon 780M Graphics; 16 logical CPUs
RAM	33,618,251,776 bytes (≈ 31.31 GiB)
Python	3.13.5
SAT	python-sat 1.9.dev16; Glucose3 (probe không có API version native riêng)
CP-SAT	OR-Tools 9.15.6755
HiGHS	highspy 1.15.1; native 1.15.1
Gurobi	gurobipy 13.0.3; native 13.0.3
CPLEX MIP	cplex 22.2.0.1; native 22.2.0.0
CP Optimizer	docplex 2.32.264; native 22.2.0.0
Ma trận sparse	SciPy 1.18.1

9.2. Dữ liệu và tập so sánh

S80 gồm 60 synthetic: bốn kích thước $N = 4, 9, 16, 25$, ba mật độ clue mục tiêu 30%, 50%, 70%, mỗi nhóm năm đề; cộng 20 public 9×9. Clue count và seed thực dùng nằm trong manifest đã khóa. Public lấy từ một file hard.txt của Sudoku Exchange, commit `d8c8ebaee0c08c412cfba96af1923dfa61c83317` [14]; seed lấy mẫu là 20261016, giữ thứ tự mẫu đã lưu.

Chọn B16 trước kết quả: trong mỗi nhóm synthetic kích thước–mật độ, sắp theo actual seed rồi instance ID và lấy đề đầu (12 đề), cộng bốn public đầu theo thứ tự mẫu cố định. C10 là phần $4\times4/9\times9$ của B16: ba synthetic mỗi kích thước và cùng bốn public. Có $C10 \subset B16 \subset S80$. SAT chạy ba encoding trên S80; CP-SAT/HiGHS dùng B16; Gurobi/CPLEX MIP/CP Optimizer dùng C10. Không chạy lại SAT riêng cho các tập con. Cấu hình chính có ba lần lặp mỗi cặp đề–cấu hình: $720 + 96 + 90 = 906$ lượt, schedule xáo bằng seed 20261016 và lưu trước chạy. Pilot đã thực thi 55 job ở lần lặp thứ nhất để kiểm tích hợp/độ phủ, không phải 906 lượt hoặc ba lần lặp chính. Không dùng tỷ lệ SAT/timeout hay thời gian pilot làm Results chính.

9.3. Đo thời gian và bảo vệ tiến trình

Worker import/đọc input trước START. START/END dùng trạng thái chia sẻ đồng bộ và đồng hồ đơn điệu. Từ START, cutoff 10 giây gồm dựng mô hình/CNF, tạo/nạp solver và solve; solve chỉ dùng ngân sách còn lại. END đúng hạn chuyển sang watchdog hậu xử lý riêng. Decode, validation, cleanup và ghi CSV/I/O đầu ra nằm ngoài build+solve. Thiếu hoặc muộn END là TIMEOUT; các thành phần thời gian chưa biết được để thiếu.

Runner Windows gán worker vào kill-on-close Job Object trước khi cho chạy, dừng process tree khi quá hạn và chờ hết descendants. Startup watchdog là 60 giây trước START; hậu xử lý là 30 giây sau END đúng hạn. Lỗi startup/hậu xử lý/cleanup là ERROR, không trộn với timeout đo. Watchdog không cấp thêm thời gian solve. Hard guard và measurement-end hook đã qua pilot trước khi dùng cho 906 lượt chính mới.

CP Optimizer dùng private preload của DOcplex 2.32.264 để tính compile/send/load vào build. Native TimeLimit nhúng trong CPO trước load; sau load, timer abort_search hợp tác được bổ sung bởi external guard. Không hứa chỉnh chính xác native TimeLimit sau load; nâng DOcplex cần kiểm lại private hook như Mục 7.

9.4. Quy tắc tổng hợp

Với mỗi đề–cấu hình đủ ba SAT, lấy median ba lần lặp cho từng thành phần build, solve và total; sau đó lấy median các median theo đề trong đúng nhóm. Ba thành phần được tổng hợp độc lập: ở mức mỗi lượt total bằng build+solve (setup nằm trong build), nhưng hai median nhóm không nhất thiết cộng thành median total. So sánh runtime cặp chỉ dùng ID đủ ba SAT ở cả hai bên, kèm matched count; solved count và timeout vẫn công bố trên toàn tập được chọn. PAR-2 theo lượt là total của SAT và $2 \times 10 = 20$ giây cho TIMEOUT; ERROR, INVALID_INPUT, UNAVAILABLE, LICENSE_LIMIT bị loại khỏi penalty. Không lấy lượt nhanh nhất, không điền timing thiếu bằng 0.

10. Thiết kế nguồn và cấu trúc dữ liệu

Kiểm tra provenance đã xác minh frozen index, manifest, witness metadata và nguồn đã pin; receipt cùng counts được lưu cục bộ. Phần này mô tả cách cấu tạo tập và giới hạn suy rộng, không suy ra hiệu năng solver.

10.1. Nguồn public, lấy mẫu và provenance

Mười nghìn đề N9 đến từ bốn nhóm file Sudoku Exchange: 2.500 easy, 2.500 medium, 2.500 hard và 2.500 diabolical tại commit `d8c8ebaee0c08c412cfba96af1923dfa61c83317` [14]. Giữ nguyên nhãn và rating nguồn, cùng URI, file/line provenance và hash nội dung. Tên nhóm là metadata nguồn chứ không là mức khó đã hiệu chuẩn cho các backend trong báo cáo. Dữ liệu nguồn được sinh bằng QQWing và chấm bởi Sukaku Explainer; vì vậy "public" không đồng nghĩa với đề do con người biên soạn hoặc mẫu tự nhiên. Khử trùng so nội dung chuẩn hóa thay vì chuỗi serialize. Audit ghi 20 ứng viên trùng/đã cũ bị loại và không còn input trùng chính xác trong tập frozen; không có overlap input với S80 hoặc pilot. Quy tắc quyền sở hữu bản public trùng ưu tiên easy, medium, hard, diabolical rồi dòng nguồn tăng dần để kết quả không phụ thuộc thứ tự hash ngẫu nhiên. Hash chứng minh byte/content identity theo chuẩn hóa được dùng, không chứng minh các puzzle không đẳng cấu.

10.2. Root synthetic và biến đổi hợp lệ

Synthetic có 5.000 đề N16 và 1.000 đề N25. Tương ứng 1.000 và 200 root cuối, cân bằng hai họ permutation và row_cycle trong từng kích thước; mỗi root cấp năm mask ở mức clue yêu cầu 20, 30, 40, 50, 60 phần trăm. Năm mức của root được giữ cùng batch để phân tích nhận ra sự phụ thuộc. Root là bảng nghiệm đầy đủ sau phép biến đổi, không là template trống hoặc bảng nguồn chung. Họ permutation chỉ dùng phép đổi nhãn giá trị hợp lệ, đổi thứ tự hàng trong band, đổi band, và phép tương tự trên cột/stack. Họ row_cycle bắt đầu từ grid đã hoán vị, chọn hai hàng khác nhau cùng band, rồi áp một chu trình proper không tầm thường lên cặp giá trị tại toàn bộ cột. Chu trình phải có độ dài 2 đến N-1; hoán đổi cả hai hàng không được tính là row_cycle. Audit ghi 600 root loại này: N16 có chu trình dài 2 hoặc 4, N25 dài 5. Các count mô tả thao tác tạo, không đếm lớp đẳng cấu. Mask giữ $\lfloor pN^2/100 + 0.5 \rfloor$ clue. Do clue count nguyên, actual N16 là 51, 77, 102, 128, 154; N25 là 125, 188, 250, 313, 375. Mật độ thực có thể lệch mức yêu cầu. Rank mask độc lập giữa mức phần trăm nên subset không bảo đảm lồng nhau. Năm mask cùng root có chung nghiệm và biến đổi; mô hình bất định phải giữ cluster root.

10.3. Witness, uniqueness và khử trùng

Metadata frozen có 16.000 witness đã kiểm độc lập trên miền, hàng, cột, khối và clue preservation. Trong đó 6.000 witness synthetic sinh cùng dữ liệu và 10.000 witness public lấy qua production solver với ngân sách acquisition riêng 30 giây. Acquisition không thuộc cutoff benchmark 10 giây và không phải kết quả của cấu hình thử nghiệm. Witness tách khỏi input solver để tránh rò nghiệm. Không có uniqueness solving mới trong audit nguồn. Public puzzles chỉ mang `source_claimed` khi nguồn có tuyên bố được dẫn; synthetic uniqueness là unverified. Một witness chứng minh tồn tại ít nhất một nghiệm, không chứng minh không còn nghiệm khác. So N9 public với N16/N25 synthetic cùng lúc đổi kích thước, nguồn và generator; không thể quy khác biệt hiệu năng chỉ cho N. Số clue không là surrogate monotonic cho độ khó.

Bảng 4
Thành phần của tập frozen.

Nhóm	N=9	N=16	N=25	Tổng
Sudoku Exchange	10,000	–	–	10,000
Permutation	–	2,500	500	3,000
Row cycle	–	2,500	500	3,000
Tổng synthetic	–	5,000	1,000	6,000
Tổng	10,000	5,000	1,000	16,000

11. Kết quả baseline ba lần lặp

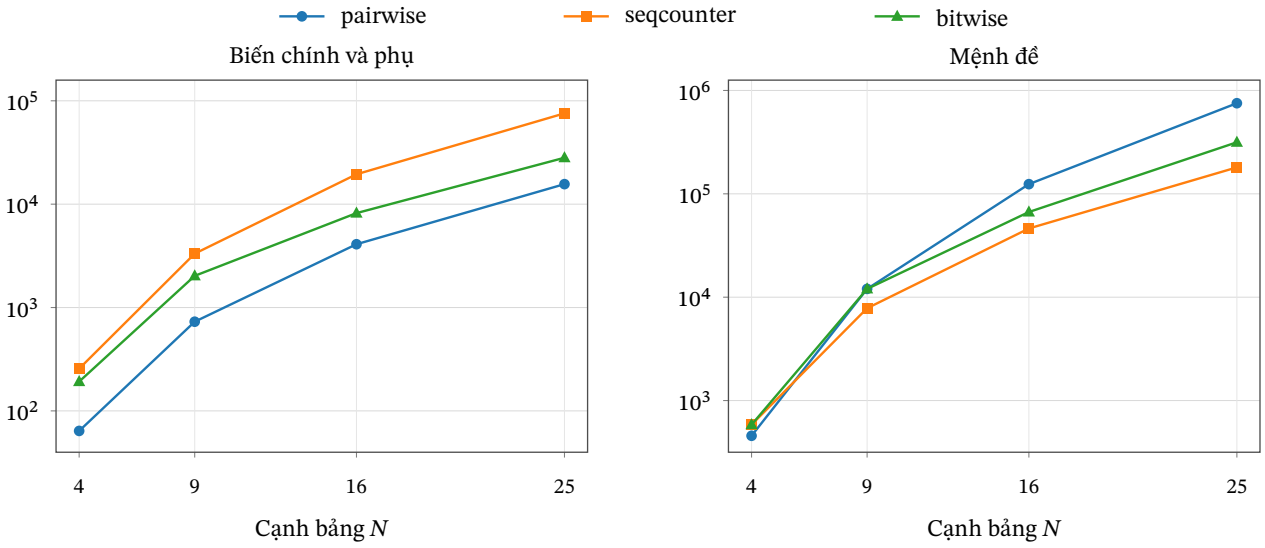
Phân tích dùng duy nhất đợt chính `main-20261016-01`: 906 lượt, 903 SAT qua checker độc lập và 3 TIMEOUT; không có UNSAT, ERROR, INVALID_INPUT, LICENSE_LIMIT hoặc UNAVAILABLE. Pilot không được gộp vào số liệu. Bảng 5 ghi cả số lượt và số đề đủ ba SAT trong đúng phạm vi.

Bảng 5
Độ phủ của 906 lượt chính.

Cấu hình	Tập	SAT/lượt	Đề đủ 3 SAT	TIMEOUT
Glucose3 pairwise	S80	240/240	80/80	0
Glucose3 sequential	S80	240/240	80/80	0
Glucose3 bitwise	S80	240/240	80/80	0
CP-SAT	B16	48/48	16/16	0
HiGHS ILP	B16	45/48	15/16	3
Gurobi MIP	C10	30/30	10/10	0
CPLEX MIP	C10	30/30	10/10	0
CP Optimizer	C10	30/30	10/10	0

11.1. Kích thước CNF và chi phí dựng mô hình

Hình 2 giữ mật độ 50% để tách ảnh hưởng của kích thước bảng; trục tung log, số biến gồm cả biến phụ, count lấy trước preprocessing. Hình dùng seed 0 đại diện vì count khớp giữa năm seed trong mỗi nhóm kích thước–mật độ. Bảng 6 minh họa riêng N25/30% với 188 clues: pairwise không có biến phụ nhưng nhiều clause nhất; sequential có nhiều biến phụ hơn bitwise nhưng ít clause hơn. Count CNF không được dùng thay kích thước native CP/MIP.



Hình 2: Kích thước CNF synthetic, mật độ clue cố định 50%; seed 0 mỗi kích thước, count trước preprocessing.

Bảng 6
CNF thực của synthetic-N25-p30-s0, 188 clues; trước preprocessing.

Encoding	Biến chính	Biến phụ	Tổng biến	Clauses
pairwise	15625	0	15625	752688
sequential	15625	60000	75625	180188
bitwise	15625	12500	28125	315188

Bảng 7 dùng cùng năm ID `synthetic-N25-p30-s0` đến `s4`, mỗi ID đủ ba SAT ở cả ba encoding. Build (gồm tạo/nạp solver) lớn hơn solve trong nhóm này. Sequential có median total thấp nhất, dù median solve của bitwise thấp hơn sequential. Vì vậy, chỉ xét solve hoặc số biến sẽ bỏ qua chi phí quan trọng của pipeline. Hình 3 giữ riêng ba mật độ; mỗi điểm là median của năm median theo đề, mỗi median đề từ ba lần lặp. Thứ tự không cố định: ở synthetic 4×4/30%, median total pairwise/sequential/bitwise lần lượt là 0.001189/0.001520/0.001582 giây. Sequential thấp nhất ở N25/30% không đồng nghĩa thấp nhất cho mọi n hoặc mọi dataset.

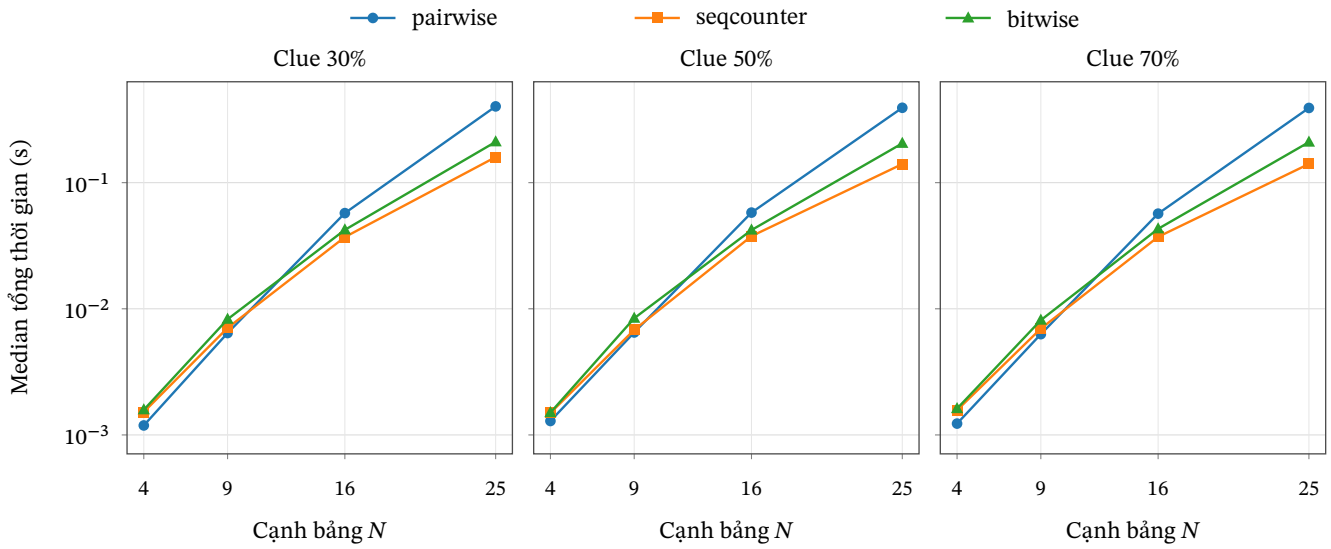
11.2. Đối chứng trên đúng tập giao

Các Bảng 8–10 lấy từ `paper/baseline-matched.csv`, dùng pairwise làm mốc cố định, không chọn encoding nhanh nhất riêng cho từng đối thủ. Median total chỉ lấy ID có đủ ba SAT ở cả hai bên. Trên B16, mỗi mật độ tại N25 chỉ có một ID, khác năm ID của thống kê S80 trong Bảng 7. Ví dụ `synthetic-N25-p30-s0`: CP-SAT 0.230552 giây so với pairwise 0.394866 giây; không thay giá trị pairwise này bằng 0.402449 giây của nhóm năm ID. Trên cùng bốn public, CPLEX MIP có median

Bảng 7
Synthetic 25 × 25, 30% clues: năm ID chung, giây.

Encoding	Median build	Median solve	Median total
pairwise	0.381882	0.022380	0.402449
sequential	0.130810	0.024383	0.159527
bitwise	0.193860	0.015416	0.210650

Mỗi cột là median của median ba lượt theo đề, tổng hợp độc lập; không cộng hai median nhóm để suy median total.



Hình 3: Median total SAT (giây), cùng năm synthetic ID mỗi điểm; ba lần lặp/ID, cutoff 10 giây gồm build (đã gồm setup) + solve. Ba encoding không có timeout.

Bảng 8
B16 synthetic: median total trên từng ID chung, giây.

N	Clues (%)	m_{CP}/m_H	CP-SAT	HiGHS	Pairwise
4	30	1/1	0.004866	0.001704	0.001184
4	50	1/1	0.004223	0.001638	0.001453
4	70	1/1	0.004099	0.001468	0.001323
9	30	1/1	0.013557	0.012148	0.006350
9	50	1/1	0.006484	0.004400	0.006544
9	70	1/1	0.005491	0.004467	0.006700
16	30	1/1	0.057879	0.047280	0.057086
16	50	1/1	0.009205	0.021862	0.058682
16	70	1/1	0.007168	0.022792	0.055755
25	30	1/0	0.230552	–	0.394866
25	50	1/1	0.055806	0.113672	0.382179
25	70	1/1	0.012724	0.066873	0.383525

m là số ID đủ ba SAT ở cả hai bên. Mỗi hàng chỉ có một ID, khác năm ID/hàng của S80. Dấu –: HiGHS có 3 TIMEOUT ở N25/30%, không có median SAT.

Bảng 9
C10 synthetic: một ID chung mỗi hàng, median total (giây).

N	Clues (%)	m	Gurobi	CPLEX MIP	CP Optimizer	Pairwise
4	30	1	0.002521	0.002137	0.029427	0.001184
4	50	1	0.002477	0.002280	0.033032	0.001453
4	70	1	0.002210	0.002362	0.030086	0.001323
9	30	1	0.010647	0.006728	0.029006	0.006350
9	50	1	0.007375	0.003787	0.029938	0.006544
9	70	1	0.007339	0.003884	0.030118	0.006700

Bảng 10
Bốn public 9 × 9 chung của B16/C10, median total (giây).

Bộ giải	Tập	m	Thời gian (s)	Pairwise
CP-SAT	B16	4	0.006506	0.006534
HiGHS ILP	B16	4	0.004704	0.006534
Gurobi MIP	C10	4	0.008019	0.006534
CPLEX MIP	C10	4	0.004748	0.006534
CP Optimizer	C10	4	0.030911	0.006534

Cùng bốn ID L069940, L293964, L269904, L215069 (prefix public-se-hard-). Pairwise là mốc cố định; không chọn encoding nhanh nhất cho từng đối thủ.

total 0.004748 giây, thấp hơn pairwise 0.006534 giây; Gurobi 0.008019 và CP Optimizer 0.030911 giây cao hơn pairwise. Đây là ví dụ chống nhận định SAT luôn thắng, không phải bằng chứng ILP hoặc một solver luôn tốt nhất. Các bảng không báo ratio; nếu cần ratio theo cặp phải lấy median của ratio từng ID, không chia hai median nhóm. Chênh lệch millisecond trên tập nhỏ cần được diễn giải thận trọng.

11.3. Timeout, PAR-2 và truy vết

Cả ba timeout là ba lần lặp HiGHS của `synthetic-N25-p30-s0`. ID này không có median SAT; tập giao runtime HiGHS với SAT/CP-SAT trên B16 còn 15 ID, còn với thương mại trên C10 vẫn 10 ID. Hai encoding SAT có 80 ID chung. Mỗi timeout nhận PAR-2 $= 2 \times 10 = 20$ giây, tổng ba penalty là 60 giây; mean PAR-2 của đúng nhóm HiGHS N25/30% là 20 giây. Không coi timing thiếu là 0: r3 thiếu cả build/solve/total vẫn để thiếu, r1/r2 có observed total khoảng cutoff nhưng vẫn TIMEOUT. Runtime của phần giải được luôn phải đọc cùng count trong Bảng 5. Các bảng/hình lấy từ analysis final đã khóa; `summary.csv`, `instances.csv`, `matched-pairs.csv` và `cnf.csv` giữ thống kê chi tiết. `paper/raw-row-index.csv` nối job ID tới dòng CSV gốc; audit giữ config, phiên bản và hash. Không gộp pilot, không chạy lại benchmark để chọn kết quả thuận lợi.

12. Kết quả đợt mở rộng, repetition 1

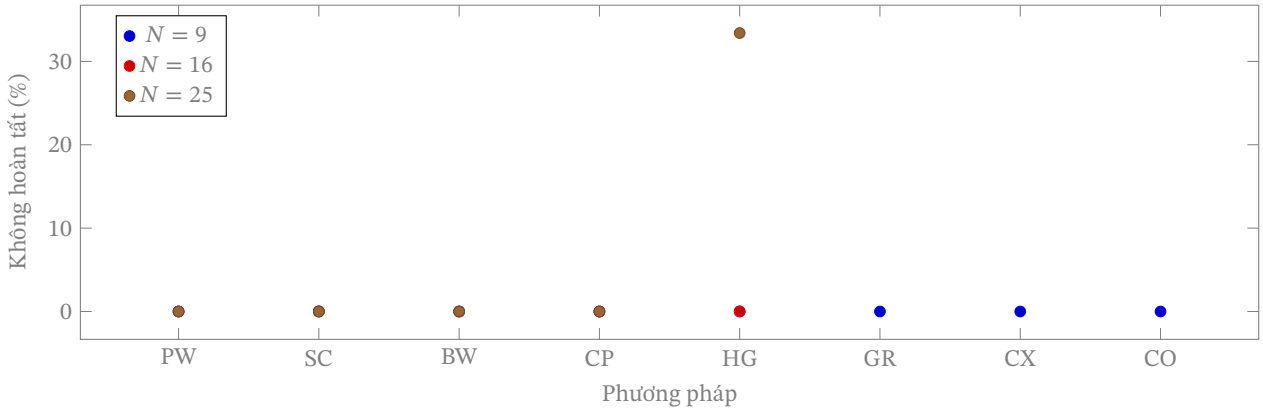
Phân tích này bao phủ 16.000 đề và đủ 110.000 lượt chính đã chọn. Có 109.664 SAT qua checker và 336 lượt không hoàn tất. Nhóm NC là cách trình bày hậu nghiệm gộp TIMEOUT và ERROR, không đổi nhãn trạng thái gốc.[1] Cả 16.000 đề có ít nhất một SAT từ một phương pháp, điều đó không hàm ý mọi solver giải từng đề. Ký hiệu bảng: PW pairwise, SC sequential counter, BW bitwise, CP CP-SAT, HG HiGHS, GR Gurobi, CX CPLEX MIP, CO CP Optimizer. Bảng 11 trình bày mẫu số theo cấu hình. Theo N, nhóm không hoàn tất là 0/80.000 ở N9, 2/25.000 ở N16 và 334/5.000 ở N25. Bảng coverage giữ mẫu số áp

Bảng 11
Coverage primary theo N và phương pháp; NC là nhóm trình bày hậu nghiệm TIMEOUT+ERROR.

N	Phương pháp	Lượt	SAT	Không hoàn tất
9	CX	10000	10000	0
9	CO	10000	10000	0
9	CP	10000	10000	0
9	GR	10000	10000	0
9	HG	10000	10000	0
9	BW	10000	10000	0
9	PW	10000	10000	0
9	SC	10000	10000	0
16	CP	5000	5000	0
16	HG	5000	4999	1
16	BW	5000	5000	0
16	PW	5000	5000	0
16	SC	5000	4999	1
25	CP	1000	1000	0
25	HG	1000	666	334
25	BW	1000	1000	0
25	PW	1000	1000	0
25	SC	1000	1000	0

dụng cho từng phương pháp. Bảng 12 dùng đơn vị đề thay vì job: mọi đề có ít nhất một primary SAT. Bảng không chứng minh toàn bộ phương pháp có cùng độ phủ trên từng đề.

[1] Trong audit, 336 lượt này gồm 335 TIMEOUT và một ERROR trước đo; chi tiết identity và bằng chứng được giữ trong phụ lục và artifact đã pin hash.



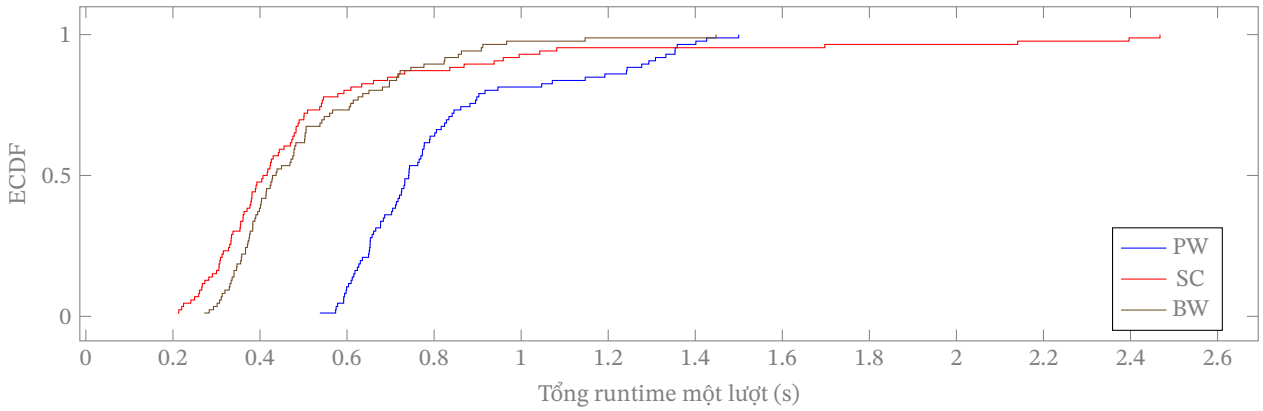
Hình 4: Tỷ lệ nhóm không hoàn tất theo kích thước và phương pháp; mẫu số là jobs áp dụng, không đo độ khó.

Bảng 12
Để có ít nhất một SAT primary theo N.

N	Đã thử	Có SAT	Không SAT
9	10000	10000	0
16	5000	5000	0
25	1000	1000	0

12.1. So sánh runtime trên tập common-SAT

Các runtime dưới đây điều kiện trên SAT và giữ đúng giao đề được chỉ rõ. Panel chính chọn session lớn nhất theo coverage, không theo tốc độ: N25/permutation/40% clue, 32 workers, profile resource-contended. ECDF, quantiles và thành phần build/solve cùng dùng 86 ID SAT chung cho ba encoding; đây là một lần đo trên từng đề, không phải median ba repetitions. PW có median 0,741841 s, SC 0,411756 s và BW 0,432963 s; p95 tương ứng 1,353671, 1,072038 và 0,897043 s. Các phân vị

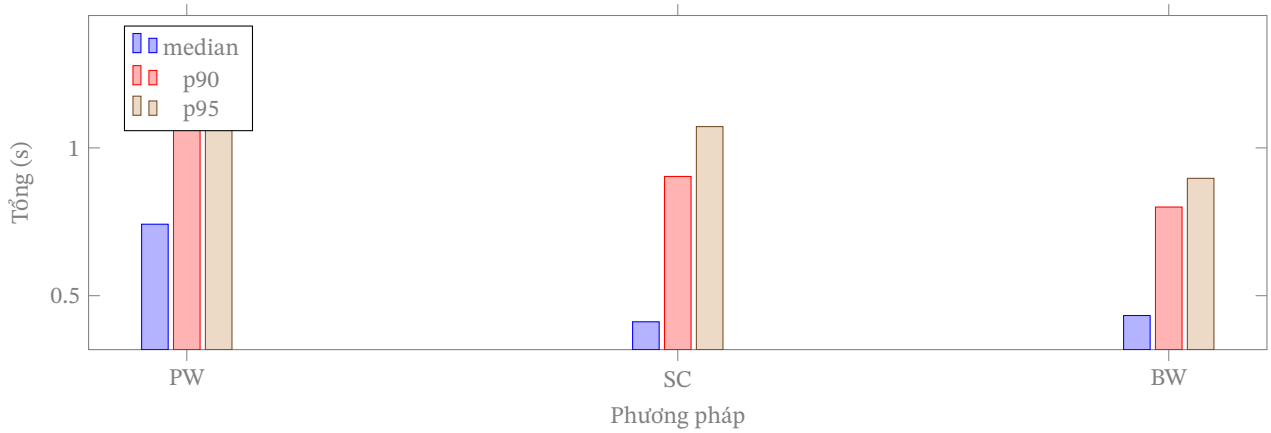


Hình 5: ECDF tổng runtime trên 86 đề common-SAT; session contended, một repetition.

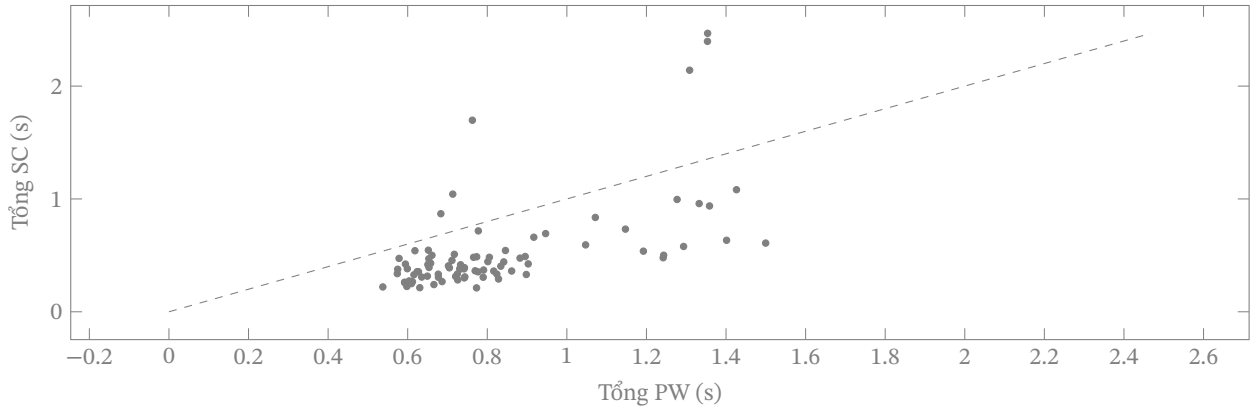
là trên đề trong một lần chạy, không phải độ lặp lại. Không có khoảng tin cậy hoặc ước lượng speedup cô lập; phiên có tài nguyên tranh chấp và nhóm synthetic chia sẻ root. Scatter dùng cặp PW/SC đã nêu riêng trong figure-scope; không suy diễn nhân quả từ session hoặc so với cấu hình khác. Các session 24/40 worker chỉ hỗ trợ coverage/timeline, không tham gia paired performance. Clue fraction không dùng làm đại diện tuyến tính cho độ khó. Hình 9 điều kiện trên SAT, tách hai family và nêu count trong dữ liệu nguồn; các mask cùng root phụ thuộc nhau.

12.2. Kích thước CNF và cấu trúc profile

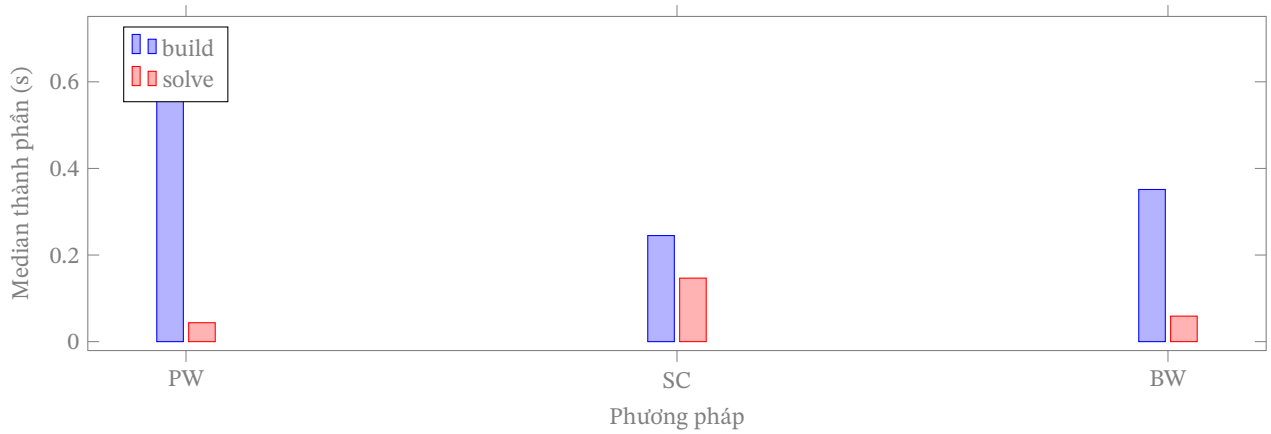
Số biến gồm biến phụ; clause counts loại clue units, trước preprocessing. Các điểm dùng cấu hình đại diện được ghi trong figure scope, không thay thế kích thước native CP/MIP. Profile timeline dưới đây là coverage theo thời điểm bắt đầu session, không phải tốc độ thực thi. Hai trial 24/40 workers được giữ để hoàn chỉnh mô tả operational coverage, không được dùng cho nhận định hiệu năng. Một repetition không hỗ trợ median ba lượt hoặc ước lượng repeatability giữa các repetitions. 324 hàng repetition 2 và 343 hàng repetition 3 trong lịch sử bị loại; diagnostic retry không được thay vào primary. Phân tích 24/40 worker không trộn vào runtime paired. Các số liệu chi tiết và raw status được giữ trong CSV được hash pin; bảng báo cáo giữ mẫu số và không nhồi toàn bộ ID.



Hình 6: Median, p90 và p95 giữa 86 đề common-SAT; phân vị mô tả biến thiên giữa đề.



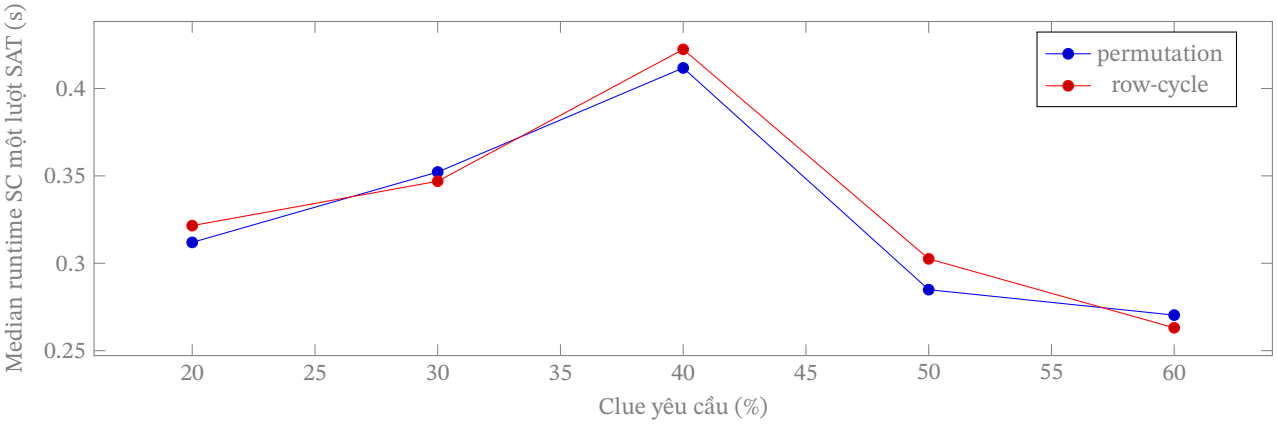
Hình 7: Runtime cặp PW–SC cho đúng 86 đề chung; đường chéo biểu thị runtime bằng nhau.



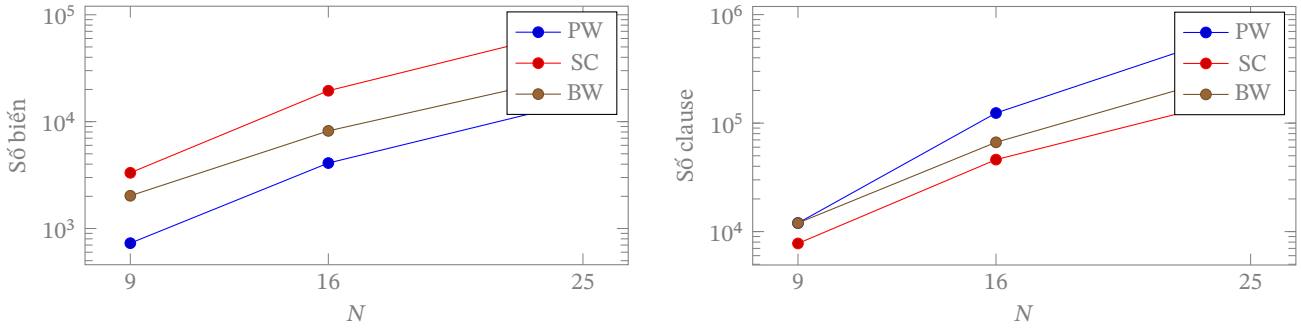
Hình 8: Median component build/setup và solve trên cùng 86 đề; hai median thành phần không cộng thành median total.

Bảng 13
Coverage theo session, profile và worker; các trial 24/40 worker chỉ báo coverage.

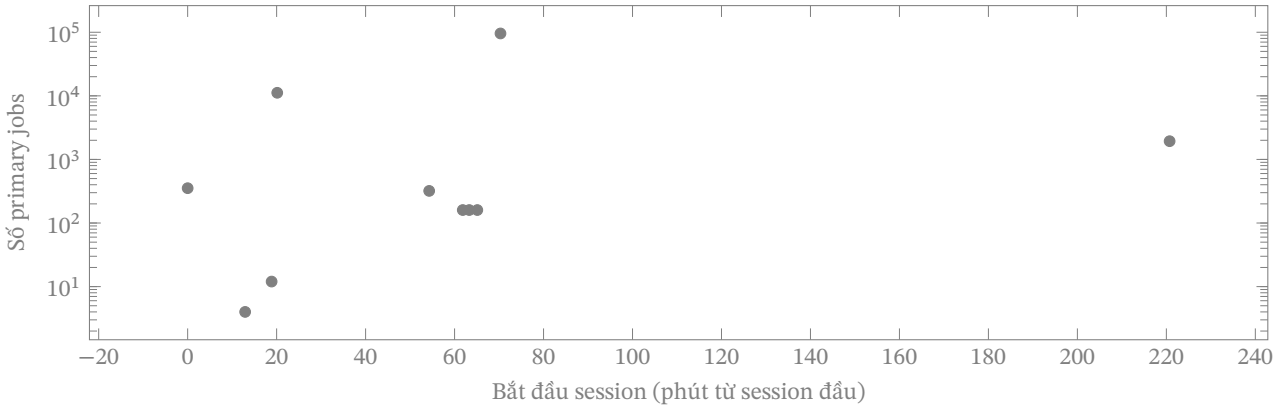
Session	Worker	Lượt	SAT	Không hoàn tất
80292632	1	353	353	0
781a1ce5	4	4	4	0
ed7d42b0	4	12	12	0
f17d0075	4	11171	11136	35
4bd685e0	8	320	318	2
d5efc155	8	160	160	0
c6569a16	24	160	159	1
b4437fae	40	160	159	1
a23e8969	32	95724	95433	291
4e0782e3	32	1936	1930	6



Hình 9: Runtime SC theo clue trong session N25; mô tả giữa các đề, không phải quan hệ nhân quả về độ khó.



Hình 10: Số biến và clause CNF theo kích thước, trước preprocessing.



Hình 11: Primary jobs theo session; trục thời gian tương đối và thang log.

13. Giới hạn nghiên cứu

Synthetic xuất phát từ bảng hợp lệ và biến thể có cấu trúc nên có generator bias, không đại diện mọi Sudoku. Lớn hơn hoặc ít clue hơn không tự chứng minh khó hơn. Public cũng do nguồn ngoài sinh bằng máy, chỉ chọn 20 đề từ một bucket hard.txt; bucket/rating là nhãn nguồn, chưa được dự án hiệu chuẩn. Bộ 80 đề có evidence SAT nhưng uniqueness chưa kiểm; tìm một nghiệm hợp lệ không phải chứng nhận duy nhất.

Đối chứng là mẫu bổ sung nhỏ: so ba SAT trên S80; với CP-SAT/HiGHS phải lọc B16; với thương mại phải lọc C10 hoặc tập giao khả dụng. So runtime cặp dùng đề có đủ ba lượt thành công ở cả hai cấu hình, công bố số đề/độ phủ đúng phạm vi. Không ghép median từ hai tập khác nhau như cùng mẫu. Timeout 10 giây là dữ liệu bị chặn theo cutoff, không phải thời gian giải thực. ERROR, UNAVAILABLE và LICENSE_LIMIT tách khỏi TIMEOUT và không tính penalty PAR-2 của timeout.

Bản miễn phí giới hạn phạm vi: Gurobi size-limited có ngưỡng 2.000 biến/2.000 ràng buộc tuyến tính; CPLEX MIP Community là 1.000/1.000. Các mô hình $4\times4/9\times9$ của C10 nằm trong ngưỡng và đã chạy đủ ba lần lặp chính trên C10; không suy độ phủ toàn S80. CP Optimizer có giới hạn riêng phụ thuộc mô hình/không gian tìm kiếm: lỗi size trên một bảng 16×16 hoặc 25×25 không chứng minh mọi bảng cùng cỡ đều không chạy được. HiGHS không được đổi nhãn thành Gurobi/CPLEX.

Một máy, một cấu hình phiên bản, ba lần lặp và cutoff hữu hạn chưa đủ khái quát cho mọi phần cứng/solver. B16 chỉ có một ID mỗi synthetic stratum; C10 chỉ có bảng nhỏ và bốn public chung. Chênh lệch millisecond dễ chịu ảnh hưởng overhead; chưa có kiểm định thống kê, chưa đo RAM hoặc search counters. Các đối chứng dùng mô hình và engine khác SAT nên không quy mọi chênh lệch cho encoding. Smoke/pilot chỉ là gate tích hợp riêng, không thuộc kết quả chính.

Phân tích mở rộng có đúng một repetition: 109.664 SAT, 335 TIMEOUT và một ERROR trong 110.000 primary jobs. Các repetitions lịch sử 2/3 (324/343 hàng) và diagnostic retry bị loại. Nhóm NC=336 chỉ là cách trình bày hậu nghiệm; raw status giữ TIMEOUT và ERROR riêng. Error guard WinError 6 xảy ra trước measurement, timing null; nó không phải lỗi cleanup. N25/permutation/40% panel là một session 32-worker có tranh chấp tài nguyên, nên không đại diện tốc độ cô lập. Các bảng ECDF, quantile và component dùng 86 đề SAT chung; phân vị là chênh lệch giữa đề của một lần đo, không phải repeatability. Pairwise scatter dùng mẫu ghép được chỉ rõ riêng. Không có CI hay ước lượng uncertainty; các mask cùng root tương quan. Thử nghiệm 24/40 worker chỉ đóng góp coverage. Độ phủ theo N9 public và N16/N25 synthetic đổi đồng thời nguồn, generator và kích thước nên không xác định tác động nhân quả của kích thước.

14. Kết luận

Mô hình Boolean ô–giá trị cho phép so sánh pairwise, sequential counter và bitwise dưới cùng đặc tả Sudoku và solver Glucose3. Baseline ba lần lặp trên 80 đề và phân tích mở rộng repetition 1 trả lời các câu hỏi ở hai mẫu số tách biệt. Trong panel mở rộng N25/permutation/40%, 86 đề SAT chung cho median runtime PW/SC/BW lần lượt 0,741841/0,411756/0,432963 giây; đây là một session có tranh chấp tài nguyên, không phải bằng chứng encoding SC luôn thắng. Build/setup là thành phần runtime riêng cần được giữ khi diễn giải. Đợt mở rộng hoàn tất identity/coverage đã chọn với 110.000 lượt primary: 109.664 SAT và 336 không hoàn tất. Nối 16.000 đề với ít nhất một SAT trên toàn bộ phương pháp không đồng nghĩa mọi phương pháp giải mọi đề. Báo cáo này không ước lượng repeatability giữa repetitions hoặc speedup cô lập. Nghiên cứu tiếp theo cần thêm repetitions và máy, kiểm uniqueness, giữ cluster root và đánh giá session không tranh chấp.

Khả dụng mã nguồn và dữ liệu

Repository công khai là `https://github.com/Ngynkybyrd/2627I_INT3011E`; baseline được công bố tại snapshot `f45d378`. Báo cáo mở rộng, snapshot, analysis và PDF của revision `final-20261020` được giữ cục bộ; các artifact mở rộng chưa public hoặc push. Manifest nguồn, receipt phân tích và hash PDF được ghi trong handoff của revision này. Mục 12 báo cáo repetition 1 của phân tích mở rộng; mục 11 giữ riêng baseline ba lần lặp.

A. Vòng đời job và bảo vệ tiến trình

Imports và input đọc trước START. Đồng hồ đơn điệu đo build/CNF, khởi tạo/nạp solver và phần solve còn ngân sách; decode, checker, cleanup là hậu xử lý riêng có watchdog. Kết quả muộn không được nhận SAT. Sau cutoff, timing không quan sát phải null, không điền bằng 10 giây. Runner gắn worker vào Windows Job Object kill-on-close và chờ process tree dừng. Watchdog không cộng thêm thời gian solve. Đường ngoại lệ dừng solve, cancel/join timer và callback, xác nhận worker đã dừng rồi mới xóa object native. Join/cleanup error cần được ghi rõ thay vì che bởi status solver. Recovery chỉ claim job chưa có primary terminal hợp lệ; ownership/fencing ngăn hai writer đồng thời hoặc worker cũ ghi muộn. Kết quả, witness/hash, attempt và terminal status cần publication nguyên tử hoặc reconciliation rõ nếu artifact ngoài database. Status là quan sát có định nghĩa: INVALID_INPUT dừng ở parser; SAT cần checker-pass; UNSAT cần bằng chứng infeasibility; TIMEOUT là cutoff trước kết quả admissible; ERROR giữ lỗi worker/guard/decode/checker/adapter. UNAVAILABLE và LICENSE_LIMIT tách riêng. Diagnostic retry không thay primary ERROR. Tách selected-scope-complete khỏi error-free vì snapshot đủ identity vẫn có thể giữ ERROR lịch sử.

B. Quy tắc tổng hợp và hợp đồng tái lập

Coverage là mẫu số đầy đủ theo cấu hình/stratum, còn runtime conditional trên SAT. Paired comparison yêu cầu cùng ID, cùng profile/session phù hợp và cả hai phía được đưa ra với count. Không chia median của hai tập khác nhau. Nếu có timeout, biểu diễn censoring và success coverage cạnh runtime; ERROR/invalid/license/missing không gộp timeout. Không gọi p90/p95 giữa instance là repeatability. Không ước lượng uncertainty coi mask/root phụ thuộc là độc lập. Receipt kết quả phải pin hash snapshot, schema/scope, mỗi artifact và analysis outputs. Reader kiểm SQLite integrity, expected-vs-observed matrix, unique identity/config, raw status, timing nullability, SAT solution/hash/checker và session metadata. Bảng được sinh từ summary đã kiểm, không tự tay sửa số. Paper consumer xác minh hash trước khi dùng; caption và prose dùng cùng denominator. Checklist phân tích cuối: identity/coverage; status SAT/TIMEOUT/ERROR; composition theo N/source/family/root/clue; encoding counts theoretical-vs-observed; successful runtime percentiles; paired IDs; timeout denominator; profile timeline; case ERROR; limitations. Mọi hàng summary truy được CSV, CSV truy được snapshot và receipt. Thiếu trường hoặc status không giải thích được thì dừng publication thay vì bỏ row. Khi tích hợp số liệu cuối, abstract, captions và kết luận dùng cùng denominator/hash. Nếu manifest ghi infrastructure error, báo quality flag đó bên cạnh coverage. Local staging không phải URL công khai.

Bảng 14
Artifact cần có trước khi tích hợp kết quả.

Artifact	Kiểm soát
SQLite snapshot	Backup nhất quán, integrity/FK, read-only hash
Receipt	Scope/schema, quality flag, artifact hashes
Run/session CSV	Identity, raw status, timings, profile provenance
Analysis summary	Denominator, censoring, matched counts, lineage
Paper source/PDF	Sinh bảng/hình từ analysis, font/layout QA

B.1. Trạng thái primary không hoàn tất

Nhóm NC trong phần kết quả là nhóm trình bày hậu nghiệm gồm đúng 335 TIMEOUT và một ERROR; tổng là 336. TIMEOUT bị kiểm duyệt tại cutoff 10 giây và không được gán runtime giả. ERROR duy nhất là lỗi guard trước measurement WinError 6, với build/solve/total null; lỗi không phải UNSAT, timeout hay cleanup failure. Diagnostic retry không thay thế primary. Bảng coverage và hồ sơ `declared-error.csv` giữ count và bằng chứng audit; status gốc vẫn tách biệt.

C. Vòng đời thực thi, ownership và phục hồi

State machine của job phân biệt PENDING, RUNNING và terminal. Claim RUNNING phải commit trước khi worker gọi solver. Result + witness/check evidence + attempt terminal phải cùng transaction DB sau execution; nếu dữ liệu ngoài DB, giao thức temp-write, fsync, atomic rename và reconciliation cần gắn receipt. Chỉ một owner được solve một job. Lease hết hoặc PID biến mất không tự chứng minh worker cũ đã dừng; recovery phải xác minh process tree và token/fence để loại late write. Nếu process dừng sau solve nhưng trước commit, observation có thể mất; protocol không tuyên bố exactly-once execution. Khi resume, terminal SAT/TIMEOUT/ERROR không chạy lại. Chỉ PENDING và orphan INTERRUPTED có thể được claim sau khi chứng minh owner/worker cũ đã chết. Mọi attempt vẫn được giữ. Sai nghiệm, witness không khớp, cleanup lỗi hoặc status ngoài taxonomy cần giữ bằng chứng rồi dừng để root phân xử, không bỏ qua để tiếp tục matrix. Snapshot final phải được tạo từ consistent backup, sau đó reader chỉ mở bản snapshot. CSV là projection có thể tái sinh chứ không là nguồn chân lý mới. Receipt nối manifest, schedule, raw database snapshot, solution/witness storage, CSV, analysis summary, bảng và hình bằng content hash. Nếu một tầng được tạo lại, lineage chỉ rõ parent hash và phiên bản tool; không sửa CSV tay hoặc cập nhật hash của output cũ bằng file mới. Diagram tách ranh giới thời gian khỏi điều kiện publish.



Hình 12: Luồng đo: checker và commit nằm ngoài cutoff solver nhưng thuộc lifecycle nghiệm thu.

Build/solve tạo raw status và timing; decode/checker quyết định nghiệm có thể chấp nhận; commit lưu một primary cùng witness và attempt. Lỗi tại một chặng không được đổi nhãn bằng cách nhìn kết quả chặng sau. Ví dụ, retry chẩn đoán SAT không làm primary ERROR biến thành SAT.

D. Từ điển trường và trạng thái

Bảng 15
Trường nhận diện và provenance dùng khi phân tích.

`instance_id`	Puzzle canonical đã freeze; giữ nguyên qua cấu hình
`configuration`	Backend và encoding cụ thể
`batch_id`	Vị trí puzzle/job trong schedule khóa trước
`session_id`	Đơn vị thực thi có receipt riêng
`profile, workers`	Profile và worker count thực tế của session
`timing_policy`	Ranh giới, cutoff và thành phần được đo
`repetition`	Selector primary; rep1 cho scope báo cáo mở rộng
`raw_status`	Kết quả trực tiếp trước taxonomy/checker admission
`build/solve/total`	Timing quan sát; giá trị chưa biết để null

Status được phân loại theo sự kiện quan sát: malformed input là INVALID_INPUT; nghiệm candidate hợp lệ và đúng hạn là SAT sau checker; chứng cứ vô nghiệm là UNSAT; cutoff đo hết trước kết quả admissible là TIMEOUT; lỗi guard, worker, decode, checker hoặc adapter là ERROR; thiếu runtime hoặc quyền thuộc nhóm unavailable/license-limited. Timeout không là bằng chứng vô nghiệm. Cleanup và measurement nên giữ status riêng nếu schema hỗ trợ. Coverage nên có target, matrix-planned, selected primary, observed primary, SAT, TIMEOUT, ERROR và status loại. Từng mẫu số trả lời câu hỏi khác nhau. Complete selection có thể gồm một ERROR và do đó không đồng nghĩa error-free. Runtime median phải ghi số SAT hữu hạn; matched ratio ghi số cặp còn lại sau lọc; denominator 0 biểu diễn null chứ không vô cực.

E. Checklist truy vết paper tới dữ liệu

1. Khớp hash và schema snapshot với receipt; kiểm SQLite integrity/FK.
2. Đối chiếu expected identity/config/repetition với toàn primary, không xóa ERROR/TIMEOUT.
3. Kiểm witness/hash/checker mỗi SAT; giữ diagnostic retry ngoài primary.
4. Lấy session/profile/worker/timing policy từ receipt, không gán hồi tố.
5. Sinh bảng coverage, composition, paired/runtime và timeout từ analysis; caption nêu đúng denominator.
6. Kiểm mọi số trong abstract/prose/conclusion khớp artifact hash; câu claim tách baseline khỏi extension.

F. Định nghĩa metric và mẫu số

Với nhóm stratum đã định nghĩa, coverage report phân loại mọi primary thành status quan sát. Tỷ lệ hoàn tất là số primary có kết quả admissible chia số primary áp dụng được trong cùng mẫu; cần trình bày numerator và denominator, không chỉ phần trăm. Nếu một configuration không áp dụng theo matrix, nó không đi vào denominator. Nếu job áp dụng nhưng ERROR, TIMEOUT hoặc không có terminal hợp lệ thì không được bỏ khỏi coverage. Runtime phân tích có điều kiện trên SAT primary đã checker-pass. Tập thành công $S_{c,g}$ cho configuration c và group g gồm đúng ID có timing hữu hạn admissible. Empirical CDF là $F_{c,g}(t) = |\{i \in S_{c,g} : T_i \le t\}|/|S_{c,g}|$. Luôn kèm $|S|$ và tổng primary group; ECDF không thể thay thế completion rate. Với một observation cho mỗi ID, median/p90/p95 mô tả khác biệt giữa puzzle. Nếu phân vị type 7 dùng, với m số quan sát sắp thứ tự và q quantile, chỉ số liên tục $h = (m-1)q$; nội suy giữa floor(h) và ceil(h). Khi m=0 kết quả không xác định; khi m=1, quantile bằng chính observation đó. Không giả mạo chính xác do báo nhiều chữ số hơn độ phân giải timing. Matched sample cho A/B là intersection ID đủ điều kiện trong cùng N/source/family/clue/profile/session rule. Ratio từng ID là $R_i = T_{A,i}/T_{B,i}$; report median R_i, count và status filter. Ratio nhỏ hơn một có nghĩa A ngắn hơn trên chính mẫu matched đó. Không chia median A và median B nếu hai median được tính trên ID khác nhau. TIMEOUT không có runtime quan sát đầy đủ. Censored runtime plot có thể hiện điểm cutoff khác biệt bằng marker riêng nếu execution record có event times, còn nếu chỉ primary terminal status tồn tại thì chỉ coverage plot phù hợp. ERROR không thuộc censored runtime vì nguyên nhân khác. PAR-2 nếu analysis dùng phải có formula, penalty, denominator, và cảnh báo rằng penalty metric không phải latency đo.

G. Threats to validity

G.1. Construct validity

Mục tiêu là tìm một completion hợp lệ, không đánh giá uniqueness, thẩm mỹ hoặc độ khó con người. Mô hình giảm bài toán thành feasibility. Ba AMO representation có cấu trúc clause khác nhau nhưng share cùng biến chính, ALO, nhóm EXO và clue order; thay đổi runtime phản ánh cả propagation và chi phí build/loading. Do đó không diễn giải clause count như trực tiếp đo độ khó hay chất lượng thuật toán. Cutoff mười giây là quy tắc vận hành, không là thời gian solve chính xác của job timeout. Total build+solve tránh loại chi phí encoding nhưng phụ thuộc API/runtime versions. Không thể đối chiếu trực tiếp với nghiên cứu chỉ tính native solve, hoặc với cutoff khác, nếu không tái định nghĩa đo lường.

G.2. Internal validity

Worker scheduling, CPU frequency, thermal limits, memory pressure, cache warmth và I/O có thể thay đổi timing. Receipt session giúp chỉ ra profile và overlap, nhưng không loại bỏ confounding. Build 4-worker có thể cạnh tranh tài nguyên; so session khác nhau là so environment/profile bundles, không tách thuần tác động thread hoặc thuật toán. Nguồn deadline event và measurement boundaries phải cùng implementation version. Lỗi guard trước START khác với timeout sau START. Muộn worker, cleanup failure, missing native timing và stale owner đều có thể làm profile observation thiếu hoặc ERROR. Primary/error lineage giúp giữ bằng chứng, trong khi retry chỉ chẩn đoán. Chỉ phân tích trên receipt đã QA mới bảo đảm dữ liệu đầu ra không che những khác biệt đó.

G.3. External và statistical validity

Một máy và một cấu hình phần mềm giới hạn chuyển giao kết quả sang phần cứng/backend khác. N9 public và N16/N25 synthetic khác generator/source; root phụ thuộc và số root ban đầu ít hơn số masks. Một repetition không cho ước lượng noise giữa runs. Vì vậy, sample size thô 16.000 không tương đương 16.000 puzzle độc lập, càng không là mẫu đại diện mọi Sudoku. Mọi phân tích family/root cần giữ grouping qua strata. Public rating chỉ dùng mô tả nhóm nguồn. Không suy nhân quả từ regression theo clue percentage nếu generator, N và family đồng biến. Phân tích tương lai có thể tăng machines/repetitions, kiểm uniqueness độc lập, đa dạng base roots, pre-register comparison strata và tách build/solve theo cùng metric. Đây là hướng tiếp theo, không là kết quả trong bản hiện tại.

H. Ma trận correctness và fault injection

Test model cốt lõi tách khỏi benchmark data. Với N nhỏ có thể vét mọi completion và so tính thỏa CNF/CP/ILP; với N=4, so projection của CNF lên biến chính với enumerator Sudoku độc lập. Điều này kiểm đúng encoding mà không chạy full benchmark. Mapping tests cover N=4/N=9/N=16/N=25 boundaries; auxiliary collision test reserve top ID; metamorphic tests đổi nhãn value/band/stack và kiểm checker invariance. Fault tests inject timer join failure, delayed callback, worker

Bảng 16
Ca kiểm cốt lõi và kết quả mong đợi ở contract.

Điều kiện	Phân loại	Bằng chứng cần giữ
Shape/miền sai	INVALID_INPUT	Không khởi tạo solver
Clue mâu thuẫn hợp lệ	UNSAT or status	Raw solver evidence
Candidate giải hợp lệ	SAT	Decode + checker receipt
Candidate vi phạm hàng	ERROR	Checker rejection
Hết cutoff trước nghiệm	TIMEOUT	Primary attempt; unknown times null
Guard trước START lỗi	ERROR	Raw error, no fake timing
Retry chẩn đoán SAT	Primary unchanged	Parent link + separate receipt

exit nonzero, process descendant after timeout, database crash before/after commit, corrupted witness hash and stale fencing token. Expected behavior is explicit status, no late publication and recoverable receipt. Tests should assert negative properties: no duplicate simultaneous solver, no terminal rerun, no SAT with rejected checker, no cleanup after timer still joined, no mutation of frozen manifest. Analysis tests riêng cover denominator, missingness, censoring, profile mismatch và root dependence. Fixture chỉ kiểm logic analyzer; nghiệm thu dữ liệu thật cần hash snapshot, schema, tất cả ID/job, witness và solution checker. Không dùng fixture hoặc pilot để lấp observed coverage.

I. Câu hỏi và caveat cho các hình kết quả

Bảng 17
Câu hỏi và giới hạn của các họ hình.

Hình	Câu hỏi	Giới hạn
Completion by N/config	Tỷ lệ status theo cấu hình?	Không là runtime ranking
SAT runtime ECDF	Đuôi runtime giữa SAT?	Conditional on success
Clue/family strata	Coverage theo cấu trúc đề?	Roots phụ thuộc
CNF size	Clause/variable tradeoff?	Không dự báo runtime
Build vs solve	Chi phí ở đâu?	Chỉ timing hợp lệ
Matched comparisons	Cặp cùng ID ra sao?	Count và profile
Session timeline	Profile/overlap thế nào?	Mô tả vận hành

Mỗi chart phải nối một câu hỏi đã định trước với đúng trường đo và denominator. Coverage bar dùng toàn primary; runtime ECDF dùng SAT-only và phải kèm solved fraction; scatter build/solve chỉ vẽ observation có hai thành phần hữu hạn; matched figure ghi matched n; timeline giữ session tách biệt; composition plot chỉ diễn tả sample/roots. Không vẽ giá trị dự kiến hoặc nội suy vùng thiếu.

J. Walkthrough từ input đến kết quả có thể công bố

Một input đi qua các bước dưới đây theo đúng thứ tự; bước sau không được sửa ý nghĩa kết quả bước trước.

1. Đọc exact bytes, xác minh source/content hash, decode schema và kiểm shape/miền. Sai input kết thúc INVALID_INPUT trước thời gian đo.
2. Chuẩn hóa clue triples nhưng giữ mapping ngược về instance ID, N, source và root. Không đoán nghiệm hoặc xóa clue xung đột.
3. Sinh model bằng builder đã pin; ghi version/config, variable top ID, clause/row count và thời điểm START/END theo timing contract.
4. Adapter gọi solver với một thread theo cấu hình, ngân sách còn lại, native cutoff/interrupt và hard worker guard. Raw result/status được giữ trước phân loại.
5. Nếu SAT, decoder chỉ đọc primary variables, tạo completion candidate rồi checker độc lập xác nhận miền, clues và từng unit.
6. Timer/callback được cancel và join; worker tree cleanup được kiểm. SAT chỉ được nhận nếu solve hoàn thành trong deadline, checker pass và cleanup/status thỏa policy.
7. Primary result, status, timing, witness/hash và attempt được commit theo contract. Analysis sau đó đối chiếu job matrix và session receipt từ snapshot.
8. Consumer sinh bảng/figures từ checked analysis, pin output hash và xác minh số xuất hiện trong PDF. Mọi lỗi reconcile chặn tích hợp.

Trình tự này phân biệt ba loại bằng chứng: model có cấu trúc đúng, solver trả status raw, và nghiệm candidate được độc lập chấp nhận. Chỉ loại thứ ba hỗ trợ claim SAT. Solver status có thể quan trọng cho UNSAT/TIMEOUT nhưng không thay checker của candidate. Report cần tránh câu rút gọn rằng “solver found SAT” nếu pipeline yêu cầu checker gate.

K. Quy trình tạo bảng và kiểm chéo

Mọi bảng kết quả có source machine-readable rõ ràng. Coverage và composition đếm từ primary snapshot, không từ danh sách file CSV hiện có trên máy. Runtime aggregate sinh từ accepted observations; paired table sinh từ intersection IDs và lọc status; profile table nối session receipt; encoding count sinh từ builder diagnostics hoặc formula đã nêu. Caption nhắc chính xác đơn vị và denominator. Các kiểm chéo độc lập gồm: tổng hàng theo status bằng observed primary; tổng coverage theo configuration khớp applicability matrix; runtime m có thể nhỏ hơn coverage SAT do thiếu timing nhưng không thể lớn hơn; paired n không vượt ID intersection sau filtering; sum composition theo source/family/N khớp unique instance set; figure points reconcile từng summary row. Định nghĩa quantile phải thống nhất giữa analysis JSON, LaTeX table và prose. Nếu một group có ít observations, nhãn count cần nằm ngay cùng kết quả thay vì trong appendix rời. Nếu zero successful timings, median không xác định; cell là null/N/A, không thay bằng zero. Các bảng nhỏ có thể làm tròn để đọc nhưng CSV giữ độ chính xác và rounding rule được ghi. Khi một error lịch sử còn trong primary, kết quả tổng hợp có thể có selected scope đủ identity nhưng không error-free. Bảng headline có thể gom status không hoàn tất theo quy tắc báo cáo đã chốt, còn audit appendix giữ raw category và receipt. Gộp để trình bày không được đổi raw status, mẫu số hay quality flag. Diagnostic retry giữ lineage riêng. Kiểm PDF bằng text extraction hữu ích để tìm số bị sót, citation/reference unresolved, nhãn table và metadata, nhưng không chứng minh bố cục. Cần kiểm render các trang có bảng dài, hình có axis labels, footer/footnote, block math và page transition. Mỗi kết luận QA cần nói rõ build ID để reviewer không nhầm source mới với PDF cũ.

L. Phạm vi so sánh và tính công bằng

SAT encoding comparison giữ solver engine Glucose3, builder conventions, clue order và model semantics chung; yếu tố thay đổi chính là AMO clauses. Tuy nhiên, auxiliary count, clause width, propagation order và data structure của library vẫn khác và là một phần của treatment. Một kết quả gắn với phiên bản PySAT/library cụ thể; không khái quát đến mọi implementation pairwise/seqcounter/bitwise. Baseline CP-SAT dùng mô hình AllDifferent; binary ILP có equality one-hot và objective zero; MIP backends nhận cùng CSR matrix, bounds và integrality. Biểu diễn nội bộ khác nhau, do đó so sánh là so pipeline thực thi đã triển khai, không phải microbenchmark thuật toán constraint propagation riêng lẻ. LP relaxation và presolve có thể làm thay đổi model trước native solve; solver logs/version hỗ trợ diễn giải nhưng không tự tạo kết quả công bằng nếu scopes khác nhau. Configuration applicability bị khóa theo N trước measurement để tránh so các engine trên mẫu chọn theo hậu nghiệm. Nhưng runtime intersection có thể nhỏ hơn coverage intersection vì TIMEOUT, ERROR hoặc missing timing. Mỗi headline runtime cần ghi thuộc tính group-level scope; không suy từ nhóm nhỏ hơn sang toàn bộ N/source. Một profile parallel có thể làm throughput toàn batch cao dù latency từng worker tăng; hai mục tiêu khác nhau. Tổng walltime includes scheduling/idle time và không bằng tổng per-job solver time. Chỉ đo event có protocol definition; không dùng throughput wall clock như runtime solver hoặc xây ranking thuật toán nếu workload overlap không cân bằng.

M. Ví dụ kiểm toán từng bước trên Sudoku 4 x 4

Phụ lục này dùng một ví dụ sư phạm, không bổ sung instance vào dataset hoặc quan sát vào benchmark. Mục tiêu là nối các công thức tổng quát với ID, clause và điều kiện kiểm tra cụ thể. Không gọi solver, không đo thời gian và không khẳng định tính duy nhất của nghiệm. Các số đếm bên dưới là kích thước công thức được xây dựng, không phải kích thước sau preprocessing.

M.1. Input, nghiệm minh họa và phép đánh số

Đặt $n = 2$, $N = 4$. Chọn bốn clue trên đường chéo và bảng hoàn chỉnh sau. Số 0 chỉ ô trống trong input; nó không phải một giá trị được mã hóa.

$$P = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad S = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 1 & 2 \\ 2 & 1 & 4 & 3 \\ 4 & 3 & 2 & 1 \end{pmatrix}.$$

Có 64 biến chính. Với hàng, cột đánh số từ 0 và giá trị từ 1, phép đánh số thuận và nghịch là

$$q = \mathrm{id}(r, c, v) = 16r + 4c + v,$$
$$t = q - 1, \qquad r = \left\lfloor \frac{t}{16} \right\rfloor, \qquad c = \left\lfloor \frac{t \bmod 16}{4} \right\rfloor, \qquad v = (t \bmod 4) + 1.$$

Tổng quát, thay 16 bằng N^2 và 4 bằng N. Ví dụ, $q = 27$ cho $t = 26$, nên $(r, c, v) = (1, 2, 3)$; đánh số lại trả về 27. Hai biên $q = 1$ và $q = 64$ lần lượt là $(0, 0, 1)$ và $(3, 3, 4)$. ID 0, ID âm hoặc ID lớn hơn 64 không thuộc miền của hàm nghịch biến chính.

Các clue được duyệt theo hàng và tạo đúng bốn unit clauses (1), (24), (44), (61). Chẳng hạn clue $(2, 2, 4)$ có ID $32 + 8 + 4 = 44$. Sự trùng giá trị 4 giữa hai clue không tự tạo mâu thuẫn: chúng nằm ở các hàng, cột và khối khác nhau.

M.2. Bốn loại nhóm có cùng literal nhưng khác ý nghĩa

Bảng sau ghi trực tiếp thứ tự literal của bộ sinh nhóm. Cột vị trí dùng chỉ số từ 0 trong danh sách 64 nhóm: 16 nhóm ô, 16 nhóm hàng–giá trị, 16 nhóm cột–giá trị, rồi 16 nhóm khối–giá trị.

Vị trí	Nhóm	Danh sách ID
6	$(r, c) = (1, 2)$	$(25, 26, 27, 28)$
22	$(r, v) = (1, 3)$	$(19, 23, 27, 31)$
42	$(c, v) = (2, 3)$	$(11, 27, 43, 59)$
50	$(p, q, v) = (0, 0, 3)$	$(3, 7, 19, 23)$

Ví dụ, nhóm cuối xét giá trị 3 trong khối trên trái; thứ tự ô là $(0, 0), (0, 1), (1, 0), (1, 1)$. Nhóm ô ở vị trí 6 có ALO $(25 \vee 26 \vee 27 \vee 28)$; nhóm hàng ở vị trí 22 có ALO $(19 \vee 23 \vee 27 \vee 31)$. Không được coi hai nhóm là độc lập về biến chính: literal 27 cùng thuộc các ràng buộc của ô, hàng, cột và khối chứa nó.

Trong S, ô $(1, 2)$ chọn ID 25, không chọn 27. Giá trị 3 của hàng 1 nằm ở ID 19; giá trị 3 của cột 2 nằm ở ID 11; giá trị 3 của khối trên trái cũng nằm ở ID 19. Mỗi nhóm trong bảng vì vậy có đúng một literal đúng. Đây là cách kiểm tra ý nghĩa nhóm, không phải kết quả suy diễn của solver.

M.3. Các clause thực tế của nhóm đầu tiên

Nhóm đầu tiên là $Z = (1, 2, 3, 4)$, tức ô $(0, 0)$. Cả ba encoding thêm ALO $(1 \vee 2 \vee 3 \vee 4)$. Ký hiệu $\overline{k}$ nghĩa là phủ định biến có ID k. Các clause dưới đây được đối chiếu bằng API xây dựng CNF hiện có, với `top_id=64`; đây không phải một lần solve.

Pairwise.

$$(\overline{1} \vee \overline{2}) \wedge (\overline{1} \vee \overline{3}) \wedge (\overline{1} \vee \overline{4}) \wedge (\overline{2} \vee \overline{3}) \wedge (\overline{2} \vee \overline{4}) \wedge (\overline{3} \vee \overline{4}).$$

Có sáu clause nhị phân, không có biến phụ và ID lớn nhất vẫn là 64. Nếu 1 và 3 cùng đúng, clause thứ hai sai ngay.

Sequential counter. API cấp các ID 65, 66, 67. Thứ tự clause thực tế là

$$(\overline{1} \vee 65), \quad (\overline{65} \vee 66),$$
$$(\overline{2} \vee \overline{65}), \quad (\overline{2} \vee 66),$$
$$(\overline{66} \vee 67), \quad (\overline{3} \vee \overline{66}),$$
$$(\overline{3} \vee 67), \quad (\overline{4} \vee \overline{67}).$$

Khi chỉ input 3 đúng, chọn $(65, 66, 67) = (0, 0, 1)$; cả tám clause đều thỏa. Khi 1 và 3 cùng đúng, 1 buộc 65 rồi 66 đúng, nhưng 3 buộc 66 sai. Việc chỉ đổi thứ tự in các clause không làm thay đổi CNF. Không nên đồng nhất cách in của API với thứ tự của công thức nguyên lý trong phần chính.

Bitwise. Hai selector là 65 và 66. Trong ví dụ này các input 1, 2, 3, 4 lần lượt yêu cầu các mã $00, 01, 10, 11$:

$$(\overline{1} \vee \overline{65}), \quad (\overline{1} \vee \overline{66}),$$
$$(\overline{2} \vee \overline{65}), \quad (\overline{2} \vee 66),$$
$$(\overline{3} \vee 65), \quad (\overline{3} \vee \overline{66}),$$
$$(\overline{4} \vee 65), \quad (\overline{4} \vee 66).$$

Input 2 và 3 cùng đúng đòi selector 65 vừa sai vừa đúng. Một input đúng xác định hai bit; nếu tất cả input sai, tám clause AMO vẫn thỏa với mọi selector, nhưng ALO sai. Do đó không được nhầm AMO với EXO.

M.4. Cấp biến phụ rời nhau và phép chiếu model

Dù nhóm đầu chỉ dùng ID 1 đến 4, chương trình dự trữ toàn bộ ID 1 đến 64 trước khi cấp phụ. Nếu bắt đầu cấp phụ từ 5, nó sẽ đụng biến chính của ô tiếp theo. Với sequential, hai nhóm ô đầu dùng các khoảng phụ 65 … 67 và 68 … 70; với bitwise, chúng dùng 65 … 66 và 67 … 68. Các khoảng được nêu riêng theo từng encoding, không thuộc cùng một CNF.

Sau mỗi họ 16 nhóm, `top_id` sequential là 112, 160, 208, 256; bitwise là 96, 128, 160, 192. Bất biến là mọi phụ mới lớn hơn top cũ và không ID chính nào bị đổi nghĩa. Tập ID dương cần để giải mã S là

$$\{1, 6, 11, 16, 19, 24, 25, 30, 34, 37, 44, 47, 52, 55, 58, 61\}.$$

Decoder được phép nhận danh sách dương này như một model bộ phận; nó không đòi liệt kê tất cả literal âm. Thêm ID phụ có dấu, chẳng hạn 65 hoặc -66, không đổi bảng giải mã. Điều đó không chứng minh các phụ ấy thỏa CNF: giải mã là phép chiếu lên biến chính, không phải bộ kiểm tra model SAT.

M.5. Bảng đếm chính xác cho bốn clue

Có $4N^2 = 64$ ALO dài 4 và bốn unit clue. Pairwise có $64\binom{4}{2} = 384$ AMO nhị phân; hai encoding kia có $64 \cdot 8 = 512$. Không loại clause trùng giữa các nhóm.

Encoding	Chính	Phụ	Tổng biến	Clause	Literal
pairwise	64	0	64	452	1028
seqcounter	64	192	256	580	1284
bitwise	64	128	192	580	1284

Cột literal đếm số lần xuất hiện, không phải số biến: ví dụ pairwise có $64 \cdot 4 + 384 \cdot 2 + 4 = 1028$. Hai tổng clause bằng nhau ở $N = 4$ không nói hai CNF giống nhau, có cùng lan truyền, hay cùng thời gian solve. Các tổng biến/clause đã được kiểm bằng `build_cnf`; tổng literal được suy từ độ dài clause và kiểm lại trên danh sách xây dựng. Tất cả là kiểm tra cấu trúc của ví dụ, không phải phép đo thực nghiệm mở rộng.

M.6. Ba tầng kiểm tra và các phản ví dụ nhỏ

Parser kiểm tra hình dạng và miền input; decoder kiểm tra cách đọc literal; checker độc lập kiểm tra Sudoku. Cả ba có mục tiêu khác nhau. Với S, mỗi hàng và mỗi cột có tập $\{1, 2, 3, 4\}$. Khối trên trái là $\{1, 2, 3, 4\}$; ba khối còn lại cũng vậy. Bốn vị trí clue giữ nguyên nên checker chấp nhận.

1. Bỏ ID 61 khỏi danh sách dương làm ô $(3, 3)$ không có giá trị: decoder từ chối. Thêm ID 2 trong khi vẫn giữ ID 1 làm ô $(0, 0)$ có hai giá trị: decoder từ chối. Thêm -1 cùng ID 1 gây dấu mâu thuẫn của cùng biến chính, cũng bị từ chối.
2. Đổi riêng $S_{0,0}$ từ 1 sang 2 vẫn cho một bảng đủ ô và có thể mã hóa thành một danh sách dương giải mã được. Tuy nhiên hàng 0 bị lặp 2 và thiếu 1; checker từ chối trước khi xét clue.
3. Đổi chỗ hai ô đầu hàng 0 giữ tập giá trị của hàng nhưng làm cột 0 thành $(2, 3, 2, 4)$, nên bước kiểm cột từ chối. Để tách lỗi khối, dùng bảng Latin có các hàng 1234, 2341, 3412, 4123: mọi hàng/cột đúng, nhưng khối trên trái chứa $1, 2, 2, 3$, nên bước kiểm khối từ chối.
4. Đổi tên toàn cục 1 thành 2 và 2 thành 1 trong S giữ tính hợp lệ của mọi hàng, cột, khối, nhưng đổi clue $(0, 0)$. Checker phải đi tới bước bảo toàn clue rồi từ chối.
5. Thay một giá trị bằng Boolean `True` không hợp lệ dù Python có thể so sánh nó bằng số 1. Parser/checker đòi kiểu số nguyên chính xác. Ngược lại, hai clue mâu thuẫn nhưng nằm trong miền không phải lỗi cấu trúc: parser vẫn nhận, để mô hình xử lý tính khả thi.

M.7. Giả mã kiểm toán và bất biến triển khai

Các bước sau tóm tắt đúng ranh giới trách nhiệm, không thêm thuật toán giải mới.

1. **Xây dựng.** Đặt top $= N^3$, danh sách clause rỗng. Với từng nhóm theo thứ tự đã định, thêm ALO, gọi AMO với top hiện hành, nối clause và cập nhật top. Cuối cùng thêm từng clue unit theo hàng. Sau k nhóm có đúng k ALO và chỉ các ID phụ đã cấp; mọi ID có trị tuyệt đối không vượt top.
2. **Giải mã.** Khởi tạo bảng 0 và bảng dấu rỗng. Từ chối literal 0 hoặc không phải số nguyên chính xác. Bỏ qua ID phụ; với biến chính, kiểm tra dấu nhất quán, rồi ánh xạ mỗi literal dương về một ô. Không cho hai giá trị khác nhau cùng chiếm ô; kết thúc phải không còn ô 0.
3. **Chứng nhận bảng.** Kiểm tra lần lượt hình dạng, kiểu/miền, tập giá trị từng hàng, từng cột, từng khối, rồi clues. Chỉ xuất bản nghiệm khi mọi bước chấp nhận. Checker không đọc encoding, biến phụ hoặc kết luận solver; vì thế lỗi đánh số/giải mã không tự trở thành chứng nhận Sudoku.

Tài liệu tham khảo

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the Twenty-First International Joint Conference on Artificial Intelligence, pp. 399–404. URL: https://www.ijcai.org/Proceedings/09/Papers/074.pdf.

[2] Eén, N., Sörensson, N., 2004. An extensible SAT-solver, in: Theory and Applications of Satisfiability Testing, Springer. pp. 502–518. URL: https://doi.org/10.1007/978-3-540-24605-3_37, doi:10.1007/978-3-540-24605-3_37. proceedings of SAT 2003; publisher citation year 2004.

[3] Google OR-Tools Team, n.d. CP-SAT Solver. Google. URL: https://developers.google.com/optimization/cp/cp_solver. living documentation; accessed 16 October 2026. Recorded OR-Tools distribution: 9.15.6755.

[4] Gurobi Optimization, LLC, 2026. Gurobi Optimizer Reference Manual. URL: https://docs.gurobi.com/projects/optimizer/en/current/reference/python/model.html. year follows the vendor's explicit 2026 citation recommendation; accessed 16 October 2026. Recorded gurobipy and native runtime: 13.0.3.

[5] HiGHS Developers, n.d. HiGHS Documentation: Python Examples. URL: https://ergo-code.github.io/HiGHS/dev/interfaces/python/example-py/. living documentation; accessed 16 October 2026. The recorded local highspy runtime is 1.15.1.

[6] Huangfu, Q., Hall, J.A.J., 2018. Parallelizing the dual revised simplex method. Mathematical Programming Computation 10, 119–142. URL: https://doi.org/10.1007/s12532-017-0130-5, doi:10.1007/s12532-017-0130-5. preferred citation recommended by the HiGHS project; not a description of its full MIP algorithm.

[7] IBM, n.d.a. CPLEX Python API Reference Manual: Meet the Python API. URL: https://ibmdecisionoptimization.github.io/cplex-python-doc/manual/03_meet_python.html. documentation banner 22.1.2; accessed 16 October 2026. API reference only: the installed cplex distribution is 22.2.0.1 and recorded native runtime is 22.2.0.0; not DOcplex.MP.

[8] IBM, n.d.b. Creating a Constraint Programming Model (DOcplex.CP). URL: https://ibmdecisionoptimization.github.io/docplex-doc/cp/creating_model.html. documentation banner V2.32; accessed 16 October 2026. Recorded DOcplex interface: 2.32.264; CP Optimizer executable runtime: 22.2.0.0.

[9] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437. URL: https://doi.org/10.1007/978-3-319-94144-8_26, doi:10.1007/978-3-319-94144-8_26.

[10] Lynce, I., Ouaknine, J., 2006. Sudoku as a SAT problem. Author-hosted paper, listed in Proceedings of AIMATH 06. URL: https://people.mpi-sws.org/~joel/publications/sudoku05.pdf. 9-page author version; author landing page lists AIMATH 06 (2006), with copyright 2005.

[11] PySAT Developers, n.d.a. Cardinality Encodings (pysat.card). PySAT. URL: https://pysathq.github.io/docs/html/api/card.html. documentation banner 1.9.dev16; accessed 16 October 2026.

[12] PySAT Developers, n.d.b. PySAT cardinality encoder source: bitwise.hh. URL: https://github.com/pysathq/pysat/blob/218c1c163f837c6512d8970e4bdf49ae02cee779/cardenc/bitwise.hh. upstream commit 218c1c163f837c6512d8970e4bdf49ae02cee779; header credits the card library from BOLT; accessed 16 October 2026. Not yet matched to the installed wheel.

[13] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. URL: https://www.carstensinz.de/papers/CP-2005.pdf, doi:10.1007/11564751_73.

[14] Sudoku Exchange, n.d. Sudoku Exchange Puzzle Bank: hard.txt. GitHub repository grantm/sudoku-exchange-puzzle-bank. URL: https://github.com/grantm/sudoku-exchange-puzzle-bank/tree/d8c8ebaee0c08c412cfba96af1923dfa61c83317. commit d8c8ebaee0c08c412cfba96af1923dfa61c83317; dataset public-domain dedication; accessed 16 October 2026. Uniqueness is a source claim, not independently verified.